

Ostbayerische Technische Hochschule Amberg/Weiden
Department of Electrical Engineering, Media, and
Computer Science

Prof. Dr. Ulrich Schäfer

Project Mobile & Ubiquitous Computing Summer
Semester 2026: BiteBound

Group 03: Simon Völkl & Johannes Schieder

Courses of study: Artificial Intelligence, Industry-4.0-Informatics

June 30, 2026



Figure 1: Project Logo of the Bite-Bound Game. Designed by Simon Völkl.



Figure 2: Waveshare ESP32-S3 1.69inch Touch Display. Source: waveshare.com

Contents

1	Introduction	3
1.1	Abstract	3
1.2	Mission Statement	3
1.3	Document Structure	3
2	Technical Concept	3
2.1	High-Level Architecture	4
2.2	Components	4
3	Hardware	4
3.1	ESP32-S3 Microcontroller	5
3.2	Concurrency	5
3.3	Sensor Data Acquisition	5
3.4	Physics Simulation & Game Logic	7
3.5	Display	8
3.6	MQTT Communication	8
4	Android App	9
5	Node-RED	10
5.1	Workflow	12
5.2	Dashboard & User Input	12
6	Project Management	13
6.1	Project Planning	13
6.2	Task Dependencies	14
6.3	Task Distribution	14
7	Discussion	15
8	Summary and Future Work	16
A	Prompts	18
B	ESP32 Code	42
B.1	esp32.ino	42
B.2	Headers	52
	References	100

1 Introduction

1.1 Abstract

Background: Designing responsive, cross-device IoT games on resource-constrained microcontrollers requires balancing low-latency local execution with secure, asynchronous network communication. **Objective:** This project presents BiteBound, a physical, tilt-controlled labyrinth game built on a Waveshare ESP32-S3-Touch-LCD-1.69 featuring real-time communication with two remote dashboards. **Methods:** The system utilizes a dual-core ESP32-S3 microcontroller to handle network communication, sensor inputs, physics simulation, game logic and display rendering, while a Kotlin-based Android app and a Node-RED interface provide bidirectional control via telemetry and command messages over the Message Queuing Telemetry Transport (MQTT) protocol. **Results:** The firmware, running on a network and game logic core, achieves a stable 50 Hz frame rate. A bakery-themed frontend provides a playful user experience, while the companion interfaces successfully allow for remote monitoring and control of the game state during gameplay. **Conclusion:** BiteBound demonstrates a robust, responsive end-to-end architecture bridging physical embedded interactions with digital companion interfaces.

1.2 Mission Statement

The objective of the BiteBound project is to implement a highly responsive, low-latency gaming platform demonstrating efficient multi-core task scheduling on resource-constrained hardware. Centered on a playful, bakery-themed aesthetic, the system requires navigating a virtual cherry (ball) through procedural mazes. The main technical challenge is maintaining a stable, high-frame-rate rendering pipeline alongside bi-directional MQTT communication, supported by native Android and Node-RED dashboards to deliver a unified, multi-platform user experience.

1.3 Document Structure

The remainder of this report is organized as follows: Section 2 outlines the high-level system architecture. Section 3 details the firmware, sensor integration, physics, game logic, and display optimizations. The companion interfaces are presented in Section 4 (Android App) and Section 5 (Node-RED). Finally, Section 6 covers project organization and milestones, while Sections 7 and 8 provide evaluation, conclusions, and future outlook.

2 Technical Concept

The technical concept of BiteBound is designed to ensure real-time responsiveness, modularity, and thread safety. By leveraging the dual-core capabilities of the ESP32-S3, the system separates latency-sensitive gameplay and rendering tasks from time-intensive network operations.

2.1 High-Level Architecture

The architecture relies on an asynchronous, event-driven design. The ESP32-S3 runs two primary execution threads across its dual cores to balance the computational workload. Core 1 is designated as the Game Core, measuring the sensor data, executing the physics engine, handling the game logic, and updating the display. Core 0 acts as the Network Core, managing secure TLS connections, MQTT messaging, and transmitting telemetry, running significantly slower than the Game Core.

Communication between the embedded hardware and the external control interfaces is bridged through the centralized MQTT broker HiveMQ [9, 19]. External clients communicate bidirectionally with the device. They send game-related commands to a dedicated topic, while subscribing to a telemetry topic to monitor real-time variables.

The system architecture is depicted in Figure 3, illustrating the interactions between the ESP32-S3 and the frontend dashboards (see `diagrams/out/`).

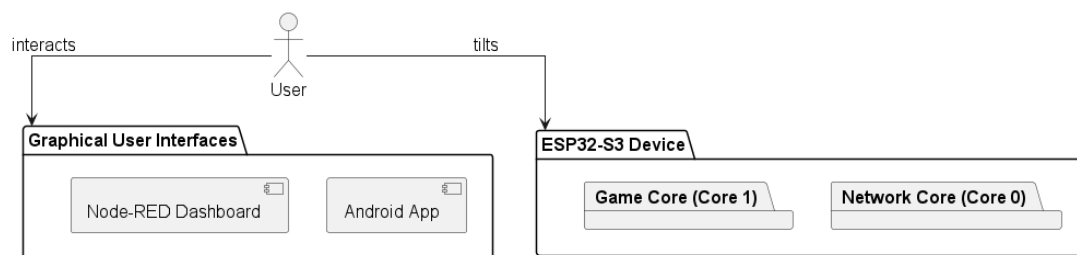


Figure 3: System architecture diagram displaying the user interactions with the ESP32-S3 and the frontend dashboards.

2.2 Components

The system architecture is divided into three primary components:

- **Hardware (ESP32-S3):** The core of the system is the Waveshare ESP32-S3 development board equipped with a 1.69-inch touch LCD, a 6-axis inertial measurement unit (IMU), and a buzzer. It hosts the procedural sensor manager, maze generator, 2D physics simulation, game engine, display logic, and network communication stack.
- **Android Application:** A native mobile interface written in Kotlin. It receives telemetry data and sends control commands to the ESP32-S3 via MQTT.
- **Node-RED Dashboard:** A flow-based UI-Builder dashboard that mirrors the application logic of the Android app.

3 Hardware

The firmware of the BiteBound platform is written in C++ using the Arduino framework [2]. The code operates on a dual-core microcontroller to separate timing-sensitive

game updates from network Input/Output (I/O) activities, as illustrated in Figure 4.

3.1 ESP32-S3 Microcontroller

The core processing unit is a Waveshare ESP32-S3 1.69-inch Touch LCD development board [30]. Onboard peripherals integrated on the board include a QMI8658 six-axis Inertial Measurement Unit (IMU), a Real-Time Clock (RTC), and an ST7789-driven 240×280 pixel Liquid Crystal Display (LCD). Dedicated hardware pins and interface buses, like Inter-Integrated Circuit (I2C) for sensors and Serial Peripheral Interface (SPI) for the display, are managed centrally via the `config.h` configuration file [31].

3.2 Concurrency

Execution is distributed across both processor cores using FreeRTOS tasks to maintain stable game loop execution rates [5]:

- Core 1 (Game Core): Runs the main application thread executing the game loop at approximately 50 Hz (20 ms loop budget). This core is responsible for sensor readings, physics steps, collision resolution, game engine updates, and display rendering. It is designed to be non-blocking and deterministic to ensure smooth gameplay.
- Core 0 (Network Core): Executes the background network task (`vNetworkTask`) at approximately 2 Hz. This core handles secure TLS handshakes, manages the MQTT client connection, and serializes and publishes telemetry data.

Inter-process communication (IPC) between cores utilizes two distinct mechanisms to ensure thread safety and avoid race conditions [6]:

- Mutex Synchronization: The global state structure (`SharedStateData`) is protected with a FreeRTOS semaphore. Access is managed through a C++ Resource Acquisition Is Initialization (RAII) helper class (`MutexLock`) to automate the lock release cycle and avoid deadlock states [4].
- Command Queue: Unidirectional commands from Core 0 are transferred to Core 1 using a thread-safe FreeRTOS queue (`commandQueue`) with a length of 10. This structure prevents network tasks from causing latency or stalling the rendering pipeline [5].

3.3 Sensor Data Acquisition

Onboard data collection is managed by the `SensorManager` class, which reads physical values from the QMI8658 IMU over I2C and samples battery voltage through an internal Analog-to-Digital Converter (ADC). The raw accelerometer (a_x, a_y, a_z) and gyroscope ($\omega_x, \omega_y, \omega_z$) readings are structured inside the `SensorData` format.

To filter out high-frequency noise and mechanical vibrations, the raw tilt inputs are smoothed using an Exponential Moving Average (EMA) filter [13]:

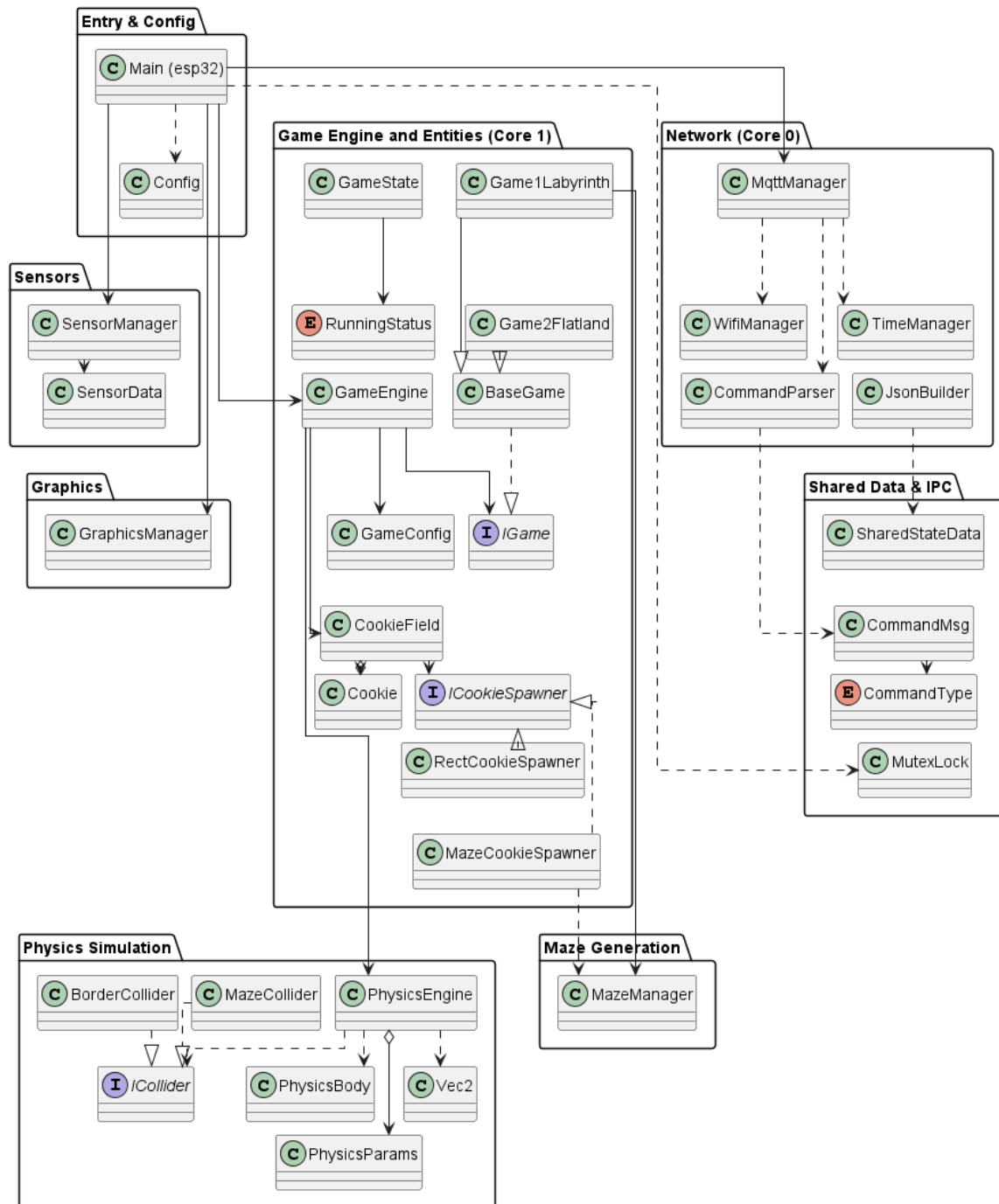


Figure 4: Hardware architecture UML diagram.

$$S_t = \alpha \cdot Y_t + (1 - \alpha) \cdot S_{t-1} \quad (1)$$

where S_t is the filtered value, Y_t is the current sensor input, and α is the smoothing coefficient (`ema_alpha` in configuration). A deadzone threshold is then applied to the output; inputs falling below this range are registered as zero, preventing drifting movement when the controller is resting.

3.4 Physics Simulation & Game Logic

The two-dimensional physics simulation is implemented inside the `PhysicsEngine` class. It is parameterized via the `PhysicsParams` structure, enabling real-time adjustments. The ball is modeled as a `PhysicsBody` with position $\vec{x}$, velocity $\vec{v}$, and radius r .

The engine uses semi-implicit Euler integration [27] with a constant time step ($dt = 0.02$ s) to update kinematics:

$$\vec{v}_{t+1} = \vec{v}_t + \vec{a} \cdot dt \quad (2)$$

$$\vec{x}_{t+1} = \vec{x}_t + \vec{v}_{t+1} \cdot dt \quad (3)$$

where $\vec{a}$ is the scaled acceleration derived from the filtered tilt values. The velocity is capped at a configurable `maxSpeed` threshold. To prevent tunneling errors when dealing with fast movements or thin boundaries, sub-stepping is utilized during collision detection.

Collisions with the environment are resolved via the `ICollider` strategy interface. The `BorderCollider` class checks for rectangular screen boundary crossings, while the `MazeCollider` evaluates pixel-level collisions against the generated layout grid. Upon detecting a collision, the engine extracts the normal vector $\hat{n}$ and calculates the updated velocity according to a restitution model [29]:

$$\vec{v}_{\text{new}} = \vec{v} - (1 + e)(\vec{v} \cdot \hat{n})\hat{n} \quad (4)$$

where $e \in [0, 1]$ represents the bounce restitution factor. The tangential velocity remains unaffected, permitting the ball to slide along walls.

Collectibles are modeled using the `CookieField` class. When the physical body of the ball overlaps with an active cookie coordinate within a configured pickup radius, the score is incremented and the `ICookieSpawner` strategy interface instantiates a new cookie. The `RectCookieSpawner` places cookies within random bounds for the open field, whereas the `MazeCookieSpawner` identifies walkable coordinates within the maze layout using generated cell lists.

The maze was generated using an iterative random Depth-First Search (DFS) algorithm [20].

3.5 Display

The rendering of the interface is performed by the GraphicsManager using the Adafruit GFX-compatible Arduino_GFX library [17].

Transmitting a full 16-bit color frame (240×280 pixels) requires writing 131.25 KB of data. Since SPI transfers one bit per clock cycle, a 27 MHz bus provides a theoretical throughput of 27 Mbit/s, resulting in a frame transmission time of approximately 40 ms, derived from the ratio between total frame size and bus bandwidth [3]. This exceeds the 20 ms limit of the 50 Hz game loop. To achieve a consistent frame rate, the display module utilizes a hybrid update scheme:

Updating a 240×280 display in 16-bit color (RGB565) requires pushing 131.25 KB of data per frame. Over a standard, stable 27 MHz SPI bus, a full frame transmission takes roughly 40 ms, which physically violates the 20ms constraint of a 50Hz game loop.

To circumvent this hardware bottleneck, the GraphicsManager implements a hybrid rendering architecture utilizing the dirty rectangles technique [26]:

- **Full Redraw:** Executed at the start of each round or when the game state changes. The manager clears the entire screen, draws the HUD, and renders the static maze layout. To optimize SPI bandwidth, horizontal spans of identical pixel states are drawn using a Run-Length Encoding (RLE) [23] approach instead of individual pixels.
- **Partial Redraw:** Used during active game updates. On every tick, the manager clears only the bounding box representing the ball's previous position. It recovers the background texture or any overlapping cookie graphics, then draws the ball at its new coordinate. This limits the active data transfer to approximately 1 KB per frame, keeping SPI communication time under 1 ms.

Volatile HUD elements, such as the score and elapsed time, are cached and updated only when their internal values change.

An example display frame of the labyrinth game is shown in Figure 5, while Figure 6 illustrates a loading screen for the game startup phase.

3.6 MQTT Communication

The network layer connects to a HiveMQ Cloud broker over an encrypted TLS connection [9]. Network credentials and SSL certificates are compiled from the git-ignored `wifi_mqtt_secrets.h` file. Certificate validation is supported by synchronization with the NTP server pool via the TimeManager during initialization.

The device establishes two main communication paths:

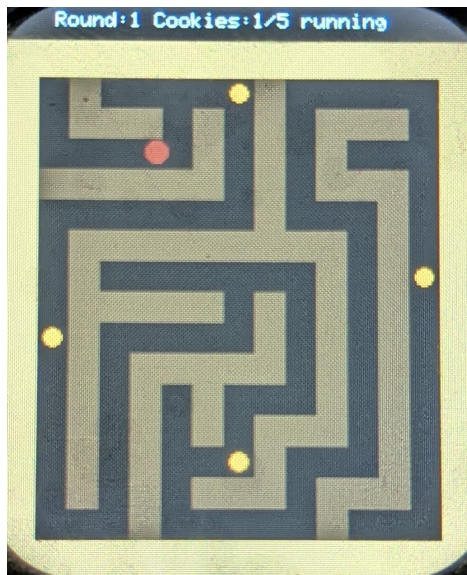


Figure 5: Example display frame of the labyrinth game on the ESP32-S3.



Figure 6: Loading screen for the game startup phase on the ESP32-S3.

- **Command Topic** (`mauc2026/group_03/game/command`): Receives control payloads from external dashboards, as illustrated in Figure 7. The incoming JSON content is processed by the static `CommandParser` class to adjust gameplay states or update physical and visual parameters on the fly [12]. A Universally Unique Identifier (UUID) [25] is used for tracking the origin of commands.
- **Telemetry Topic** (`mauc2026/group_03/game/telemetry`): Published telemetry data to dashboards at a 2Hz rate, as shown in Figure 8. The `JsonBuilder` class structures the combined hardware information, game progress, configuration records, physics parameters, and raw sensor readings into a unified JSON format.

All communication packets default to Quality of Service 1 (QoS 1) with the retain flag set to false to avoid processing stale telemetry.

4 Android App

The BiteBound Android application acts as a remote controller and telemetry dashboard, facilitating secure, real-time bidirectional communication with the ESP32-S3 over MQTT. Developed in Kotlin [11] using the Model-View-ViewModel (MVVM) architecture [16], the system separates data and transmission processes from user interface rendering across four architectural layers:

- **Data Layer:** Manages profile persistence via Android `SharedPreferences` [8], telemetry deserialization, command serialization, and local data structures. This layer matches configuration constants, boundaries, and different ball presets to the ESP32 firmware specifications.

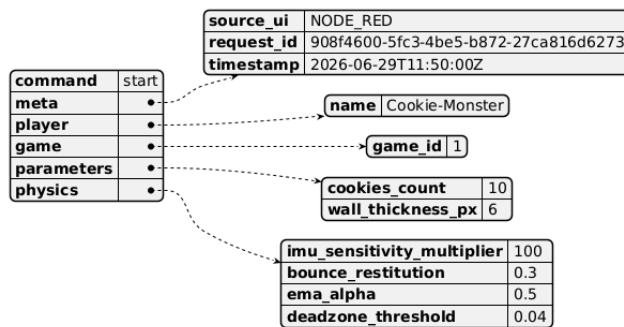


Figure 7: Dashboard JSON Command Payload to the ESP32-S3.

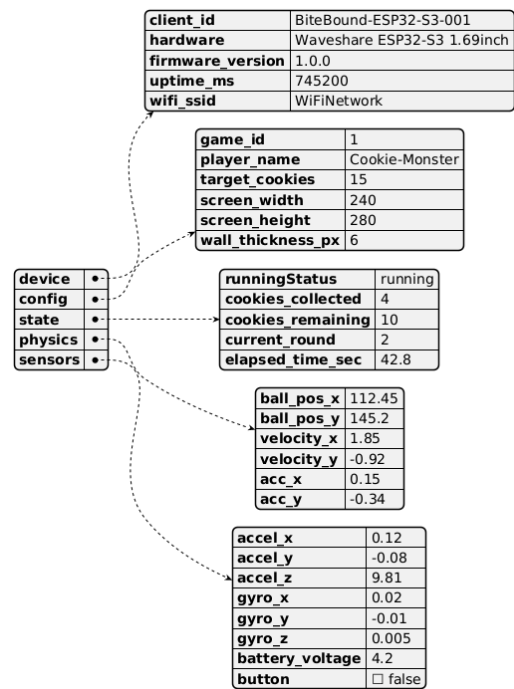


Figure 8: ESP32-S3 JSON telemetry payload to the dashboards.

- **Network Layer:** Handles connectivity tasks using the HiveMQ client [10]. It coordinates SSL/TLS handshakes [1], topic subscriptions, automatic reconnection cycles, and message publication.
- **UI State and Logic Layer:** Orchestrated by a central ViewModel that maintains screen states, parses real-time telemetry updates, monitors connection heartbeats, and dispatches command requests.
- **UI Layer (Screens):** Renders the interface using declarative, stateless composables via Jetpack Compose [7]. It features a configuration screen to initialize parameters (Figure 9) and an active gameplay dashboard that displays performance metrics as a radial progress bar [24], sensor diagnostics, and a dynamic canvas [28] tracking ball position and velocity (Figure 10).

5 Node-RED

Node-RED [22] serves as a desktop-accessible companion to the Android application [22], providing alternative telemetry processing.

This system utilizes the code-first UIBUILDER package [14] rather than the officially deprecated dashboard [21] as the latter introduces potential security and support liabilities in modern Node.js runtimes. Additionally, UI Builder supports high-frequency telemetry streams by decoupling the client-side representation from the

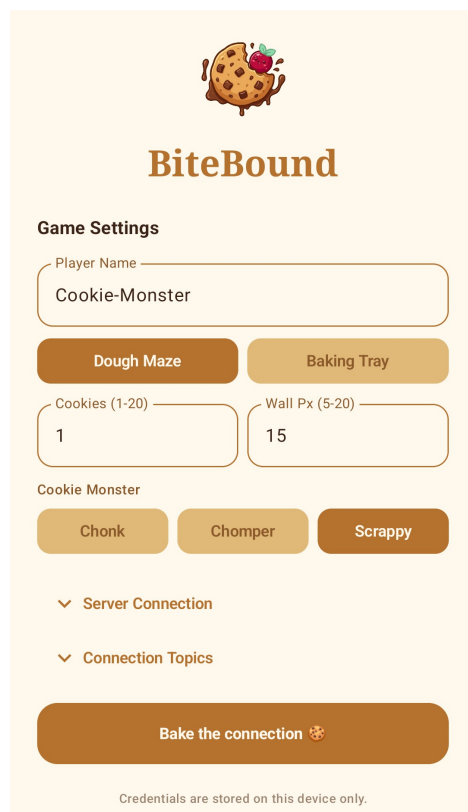


Figure 9: Android app connection setup screen for MQTT broker and game configurations.

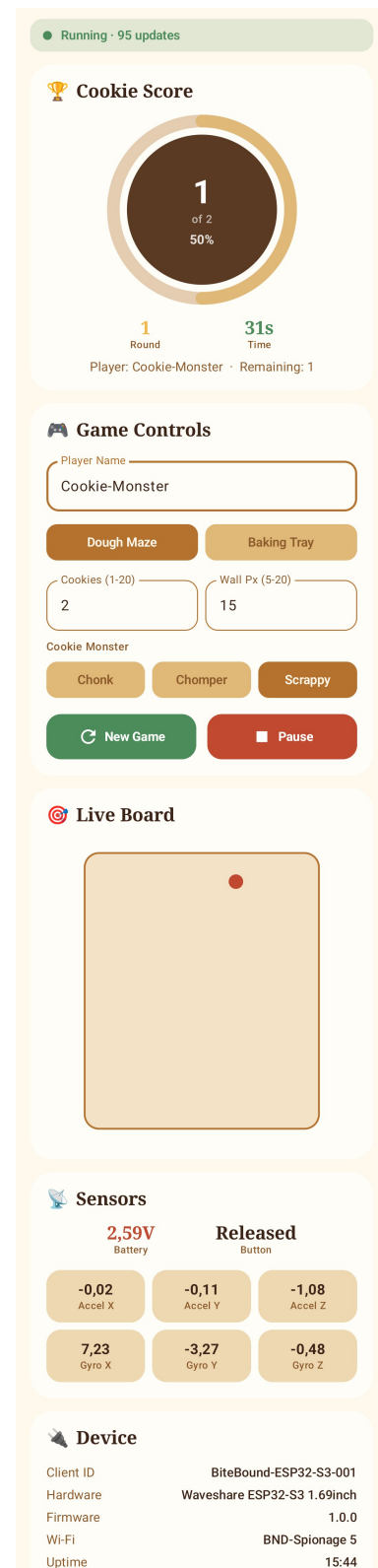


Figure 10: Android app dashboard displaying real-time telemetry and controller interface.

Node-RED backend using optimized WebSockets [18].

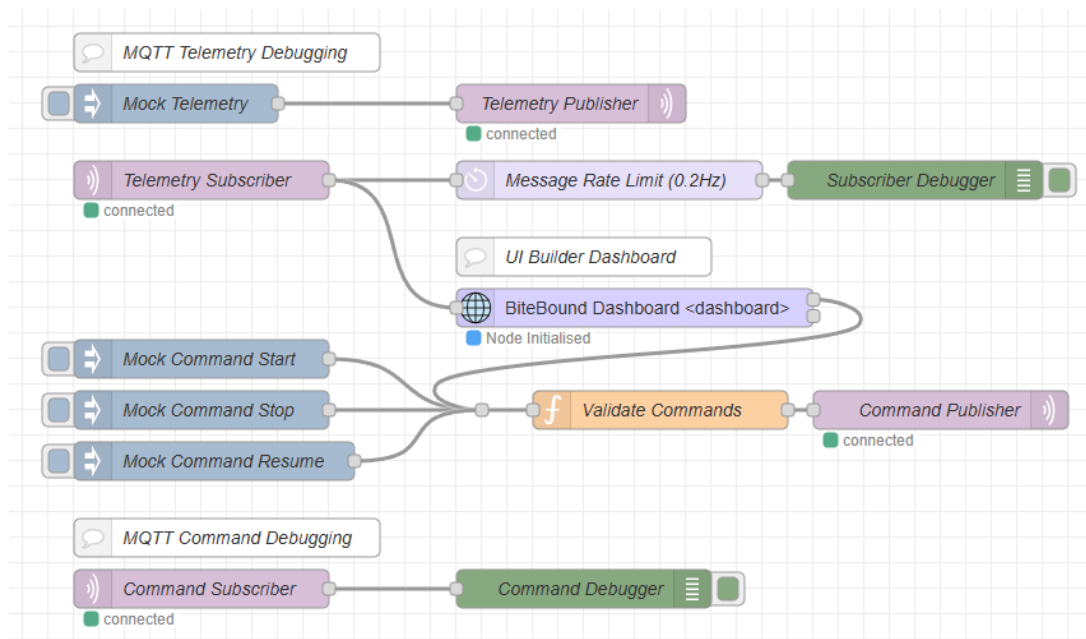


Figure 11: Node-RED backend flow orchestrating MQTT communication and dashboard integration.

5.1 Workflow

The backend flow orchestrates MQTT setup, topic routing, debug logging, and state initialization, as depicted in Figure 11. Incoming telemetry is pipelined directly into the BiteBound Dashboard uibuilder instance, while a parallel logging branch uses a filtering delay node to limit outputs. Manual command injections are available to bypass the UI for debugging purposes.

5.2 Dashboard & User Input

The uibuilder interface, as illustrated in Figure 12, offers a responsive, desktop-optimized web console that mirrors the style, capabilities and functionality of the Android app. The interface is modularized within the local web directory, separating markup, styling, and application logic:

- `index.html`: Establishes the DOM structure, dividing the workspace into cards.
- `index.css`: Implements a uniform, cookie-themed color palette using CSS variables to maintain styling coherence.
- `index.js`: Acts as the script entry point, initializing the client-side uibuilder library and listening for active WebSocket [15] updates from the flow.

The core logic processes telemetry inputs into standardized JavaScript objects, validates them against JSON specifications, and constructs device commands for Web-

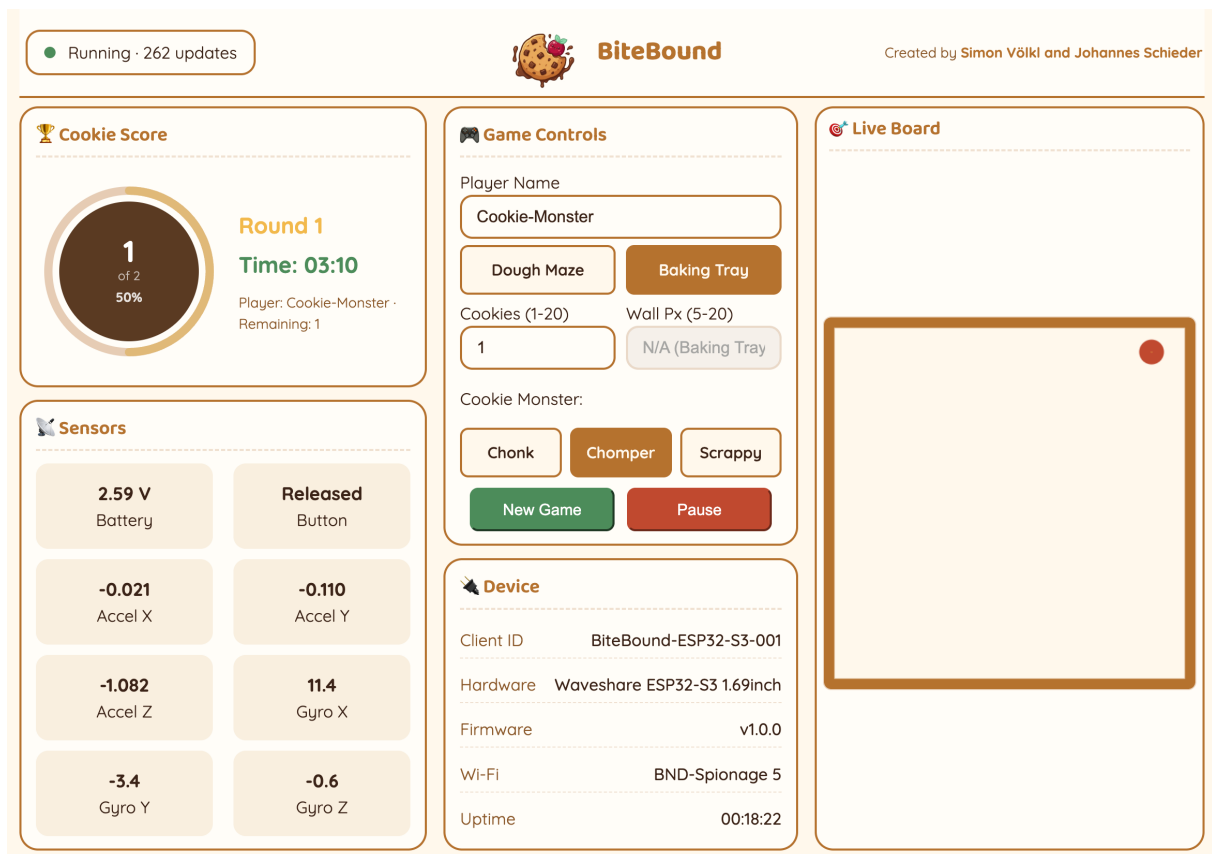


Figure 12: Web-based operator dashboard rendered via uibuilder.

Socket transmission. DOM rendering dynamically updates the HTML content based on the current state, though no data persistence is implemented.

6 Project Management

The BiteBound project was executed by a two-person team, with each member contributing to all aspects of the project. The lead developers for each component are indicated in Table 1, while a reviewer was assigned to each task to ensure code quality and maintainability, as stated in Section 6.3.

6.1 Project Planning

The development of BiteBound followed an incremental path, prioritizing the definition of interface contracts and network infrastructure before implementing complex local logic such as the physics simulation and graphics rendering. The project was structured into four distinct milestones to manage dependencies and verify integration at each phase.

- **Milestone 1: Architecture and Basic Setup** This phase focused on creating the

repository structure, planning extensive software architecture, establishing development environments, and configuring the basic ESP32-S3 firmware skeleton. Essential network modules (WiFi connectivity, secure MQTT communication via HiveMQ Cloud, NTP-based time synchronization) were implemented. Defining the standard JSON schemas for telemetry and command topics early in the timeline allowed the basic setups of the Android application and Node-RED dashboards to proceed in parallel with the firmware development.

- **Milestone 2: Hardware Implementation** During this milestone, the core offline mechanics of the ESP32 firmware were established. The 2D physics engine and cookie collection logic were designed first, followed by the integration of the Sensor Manager and maze generation. The graphics subsystem was then built, including an evaluation of display libraries. Finally, the firmware was fully orchestrated by the combination of the Game Engine for the game modes and the FreeRTOS-based dual-core task management.
- **Milestone 3: UI Enhancement and Refinement** This phase concentrated on system integration, user interface polishing, and debugging. The Android application and Node-RED dashboards were updated to support the finalized game orchestration logic, enabling dynamic command and parameter handling. The project's visual identity with the logo, color scheme and assets was finalized, while last tests and bug-fixes were conducted to address firmware issues.
- **Milestone 4: Documentation** The final phase was dedicated to finalizing the technical report as documentation, enhancing the usage guide, and preparing the project for submission.

6.2 Task Dependencies

The decoupled software architecture introduced several critical dependencies: IMU calibration fed into physics input, JSON schema definitions enabled dashboard integration, and dual-core allocation required thread-safe state protection before merging the network and game codebases. These dependencies ensured that network managers and telemetry contracts were established first, physics and sensor processing were validated before graphics integration, and the multi-core architecture was introduced only after both stacks were independently stable.

6.3 Task Distribution

To coordinate development and ensure progress, tasks were tracked and managed using a GitHub Project board integrated directly with the code repository. This allowed issues to be linked to commits and Pull Requests (PRs).

To maintain code quality and structural integrity across the shared codebase, the development followed several guidelines:

- **Branch Protection:** Direct commits to the main branch were restricted. All feature updates and bug fixes were developed on separate branches.

- **Code Review:** Merging any branch into main required a Pull Request accompanied by the other team member's review and approval.
- **CI Pipeline:** A GitHub Actions pipeline was configured to automatically verify the compilation of the ESP32 firmware for both test and production builds (see `.github/workflows/ci.yaml`)
- **Code Coverage Targets:** A unit test coverage of about 60% was targeted for the core ESP32 modules, while the Android application aimed for basic unit tests. Node-RED flows were excluded from automated testing due to their visual nature (see `esp32/test-coverage.txt`).
- **Docstrings:** All public methods and classes were documented with docstrings to enhance code readability and accessibility.

Transparency note: The unit test writing and docstring generation were partly assisted by AI, as documented in Section A. The division of responsibilities between the two team members is outlined in Table 1.

7 Discussion

During the development of BiteBound, several technical challenges were encountered, each requiring specific solutions to meet the set requirements.

As already mentioned in Section 3.5, the SPI transmission overhead posed a significant challenge to maintaining a stable 50 Hz frame rate. By implementing a hybrid rendering system using the dirty rectangles technique, a substantial reduction in payload size was achieved, allowing for faster transmission times.

Recursive algorithmic approaches could lead to stack overflow errors without setting maze size limits. Therefore, the iterative DFS implementation for maze generation was a necessary design choice to generate arbitrary maze sizes without limiting the system to an upper bound.

Sensor value noise and drift were expected at the beginning of the project. Implementing a deadzone threshold and an EMA filter provided a straightforward solution to ensure stable physical interactions while still allowing for raw telemetry data to be available for diagnostic purposes.

Testing the physics engine after implementation revealed that at high tilt angles, the ball could tunnel through maze walls due to its accelerated velocity. The implementation of sub-stepping during collision detection effectively mitigated this issue, ensuring that boundaries were respected even at high velocities, as described in Section 3.4.

During the architecture phase, the assumption was made that handling incoming dashboard commands and running the game loop on one core could cause significant frame drops. Early in development, the two-core possibility was explored, included in the initial design, and later implemented (see Section 2). This decision proved crucial

for the modular system design and, consequently, the testability, maintainability, and scalability of the system; the dual-core synchronization strategy worked as expected.

As only one LCD display was available, the other team member had to acquire a separate ESP32-S3 board to test and run the hardware code. Though, the other board only had one core, which limited the ability to test the dual-core synchronization strategy.

The HiveMQ broker was found to be inconsistent when handling multiple subscriptions simultaneously, leading to lags and disconnects. Still, no other broker was chosen for further development, as this issue only appeared sporadically.

8 Summary and Future Work

The BiteBound project demonstrates a practical approach to addressing the primary objectives established in Section 1, specifically managing low-latency gameplay execution alongside secure, asynchronous network communication on resource-constrained hardware. By partitioning the timing-sensitive physics engine and rendering pipeline onto the Game Core and the communication onto the Network Core (Core 0), the system maintains a stable frame rate. This separation of concerns prevents network latency from interrupting physical gameplay while maintaining bidirectional synchronization with the Android and Node-RED companion interfaces, validating the multi-platform user experience proposed in the mission statement.

Future developments will focus on expanding the platform's physical interactivity, sensory feedback, and academic dissemination. To enrich the gameplay mechanics, upcoming iterations can incorporate onboard physical button inputs to trigger special, context-dependent actions during active play. Integrating a vibration motor or haptic sensor would further ground the physical interaction by providing physical feedback when the virtual ball collides with maze boundaries. Auditory feedback can also be expanded by implementing driver routines for the onboard buzzer, generating custom startup sequences, loading music, and distinctive alerts for coin-collection events. Finally, this technical report will be formatted into a scientific paper suitable for public submission to relevant mobile and ubiquitous computing venues, allowing for broader academic evaluation of the architecture.

Table 1: Task Distribution Matrix Grouped

Milestone	Task	Description	S. Völkl	J. Schieder
Milestone 1: Architecture & Setup	Architecture	Software Architecture, Protocol Definitions, Repository Setup	Lead	Lead
	Networking	Implementation of WiFi, MQTT and Time synchronization.	Lead	Review
	Android App	Basic Android app, MQTT transport via HiveMQ.	Review	Lead
	Node-RED	Basic Node-RED setup, UI-Builder Dashboard.	Lead	Review
Milestone 2: Hardware Implementation	Tests	Automated AUnit Tests for hardware components.	Lead	Lead
	Physics	Core 2D physics engine, collisions, and cookies.	Review	Lead
	Sensor Manager	Configuration of QMI8658 IMU, battery and button inputs.	Lead	Review
	Maze	Iterative random DFS.	Lead	Review
	Graphics	Library comparison, Driver setup, ST7789 config, rendering.	Lead	Review
	JSON Builder and Parser	JSON serialization of game state and deserialization of commands.	Lead	Review
	Concurrency	FreeRTOS multi-core tasking, queues, and mutex sharing.	Lead	Review
	Game Engine Game Orchestration	Game modes, engine and start. Orchestrated concurrency and game engine in Arduino.	Review Lead	Lead Lead
Milestone 3: UI & Refinement	Branding	Designing the logo, color scheme, and visual assets.	Lead	Lead
	Enhanced Android App	Updating Android App to support new features and improvements.	Lead	Lead
	Enhanced Node-RED UI	Updating Node-RED UI to support new features and improvements.	Lead	Review
	Manual Testing	Manual code testing and bug fixes.	Lead	Lead
Milestone 4: Documentation	Technical Report	Finalizing the LaTeX report, code docs, and usage guides.	Lead	Review
	Presentation	Preparing and delivering the final presentation.	Lead	Lead

A Prompts

For transparency and reproducibility, the used AI tool prompts are attached in this section. In general, the model Google Gemini Flash 3.5 Thinking was used for all prompts.

UI State Persistence Guide

Prompt:

I am developing the 'BiteBound' Android app using Kotlin and Jetpack Compose. I need to manage local UI state for graphical elements across recompositions. Please provide a professional guide on using `remember` for UI state persistence, including examples showing basic usage for graphics.

Answer:

In Jetpack Compose, `remember` retains objects across recompositions. While `BiteBoundViewModel` handles complex logic, `remember` manages transient graphical states.

Basic usage:

```
@Composable
fun GraphicComponent() {
    // Persists state across standard recompositions
    var isActive by remember { mutableStateOf(false) }

    Button(onClick = { isActive = !isActive }) {
        Text(if (isActive) "Active Graphic" else "Inactive Graphic")
    }
}
```

Use `rememberSaveable` to survive configuration changes like rotations.

Compose Cookie Theme Generation

The generated code was directly implemented into the Android Frontend.

Prompt:

As an expert Android developer, please design a Jetpack Compose theme for a bakery app called 'BiteBound'. The design system should use a cookie-inspired color palette with warm tones like 'CookieDough' and 'CaramelBrown', supporting both light and dark modes. Map these to a Material 3 `lightColorScheme`. Additionally, configure standard Typography using a custom 'Display' font for headlines. Place the files in the `theme` folder, and briefly explain how the light and dark modes are handled.

Answer:

I have generated the Jetpack Compose design system for BiteBound. The resulting files are located in the ``com.example.bitebound.ui.theme`` folder.

Summary of the generated architecture:

1. `Color.kt`: Contains the raw cookie-themed color palette (e.g., `CookieDough`, `CaramelBrown`) and their dark mode counterparts (e.g., `NightDough`).
2. `Type.kt`: Configures the Material 3 Typography, utilizing your custom `'Display'` font for `headlineLarge` and `headlineSmall`.
3. `Theme.kt`: Implements the `MaterialTheme`, defining both `lightColorScheme` and `darkColorScheme`.

The app will seamlessly transition between light and dark mode by evaluating the system preference and applying the respective color scheme. To utilize basic graphics and colors in your UI, wrap your screens in the generated theme wrapper and apply modifiers such as ``Modifier.background(MaterialTheme.colorScheme.primary)``.

Architecture Design Proposal

Prompt:

I need a professional system architecture proposal for an embedded IoT project called "BiteBound".

Hardware: Waveshare ESP32-S3 1.69" Touch Display Board (ST7789V2 LCD, QMI8658 IMU).

Firmware: Arduino framework (C++) with GFX library to build two tilt-controlled games (a random maze game and an open-arena ball game with wall reflections).

Connectivity: Bidirectional MQTTS via HiveMQ Cloud.

Dashboards: Synchronized Node-RED web dashboard and an Android app.

Requirements:

- High-level architecture design separating game loops and network tasks.
- Clean project folder structure for submission.
- Bidirectional MQTT communication layout using JSON payloads (telemetry from ESP32, configuration commands from dashboards).
- Brief mention of core algorithms needed for the 2D physics engine and maze generation.

Please provide a structural overview only -- do not generate any implementation code yet.

Answer:

SYSTEM ARCHITECTURE PROPOSAL: PROJECT BITEBOUND

Target Hardware: Waveshare ESP32-S3 1.69" Touch Display Board

1. SYSTEM OVERVIEW

Project "BiteBound" is an IoT-enabled, tilt-controlled physical gaming platform built on an embedded microcontroller architecture. Utilizing an onboard IMU for precise tilt-sensing and a round/rectangular TFT display for visualization, the system offers real-time physics and maze-based gameplay. Concurrent bidirectional cloud communication via MQTTS enables remote dashboard monitoring, operational synchronization, and over-the-air parameter configuration.

2. HIGH-LEVEL ARCHITECTURE (FIRMWARE & NETWORK SEPARATION)

To maintain deterministic 60 FPS gameplay while managing non-blocking network I/O, the firmware exploits the dual-core architecture of the ESP32-S3 via FreeRTOS.

Core Allocation Strategy

- * **Core 1: Real-Time Game & Perception Loop (High Priority)**
 - * **Responsibilities:** IMU sensor polling, 2D physics integration, collision detection, maze wall logic, and high-speed GFX frame-buffer rendering.
 - * **Execution Model:** Fixed time-step loop (e.g., 16.6ms for 60Hz target) to guarantee consistent physical interactions.
- * **Core 0: Network & Asynchronous Telemetry Loop (Low-to-Medium Priority)**
 - * **Responsibilities:** Wi-Fi connection state machine, Secure MQTT (MQTTS) keep-alives, inbound JSON command parsing, and outbound JSON telemetry serialization.
 - * **Execution Model:** Event-driven/blocked state, awakening upon received network packets or when notified by Core 1 telemetry triggers.

Inter-Task Communication (ITC)

- * **Telemetry Queue (Core 1 -> Core 0):** A thread-safe FreeRTOS Queue passing bounded structs containing current game states, scores, and active frame metrics.
- * **Configuration Mutex/Shared Structure (Core 0 -> Core 1):** A shared configuration block protected by a lightweight Mutex, updated when dashboards publish new gameplay variables (e.g., gravity scalar, difficulty).

3. PROJECT FOLDER STRUCTURE

A standardized, modular PlatformIO directory layout ensures clean encapsulation of drivers, business logic, and assets.

```
\texttt{bitebound/}
\begin{itemize}[label=]
\item firmware/ -- ESP32-S3 Arduino C++ Project
  \begin{itemize}[label=]
    \item include/ -- Headers
    \begin{itemize}[label=]
```

```
\item config.h, game\_engine.h, maze\_gen.h, mqtt\_handler.h, periph\_imu.h
\end{itemize}
\item src/ -- Implementations
\begin{itemize}[label=]
\item main.cpp, game\_engine.cpp, maze\_gen.cpp, mqtt\_handler.cpp, periph\_imu.cpp
\end{itemize}
\item platformio.ini -- Dependencies
\end{itemize}
\item dashboards/
\begin{itemize}[label=]
\item node-red/ -- flows.json
\item android/ -- Native app
\end{itemize}
\item docs/ -- Architecture schemas
\end{itemize}
```

4. BIDIRECTIONAL MQTT COMMUNICATION LAYOUT

All messaging routes securely through HiveMQ Cloud over TLS (Port 8883) using JSON string payloads.

Topic Namespace Strategy

- bitebound/{device_id}/telemetry -- Published by ESP32-S3 to Cloud (Interval: 1 Hz or on Game Over)
- bitebound/{device_id}/config -- Subscribed by ESP32-S3; Published by Node-RED/Android App

Payload Data Schemas

A. Telemetry Schema (Device -> Cloud)

```
{
  "timestamp": 1782631200,
  "device_id": "esp32s3_bb_01",
  "status": "PLAYING",
  "active_game": "OPEN_ARENA",
  "metrics": {
    "score": 1420,
    "level": 3,
    "elapsed_sec": 45.2
  },
  "dynamics": {
    "tilt_x": -0.12,
    "tilt_y": 0.45,
    "ball_vel_x": 3.4,
    "ball_vel_y": -1.2
  }
}
```


B. Configuration Command Schema (Cloud -> Device)

```
JSON
{
  "command_id": "cmd_9921",
  "target_game": "MAZE",
  "settings": {
    "physics_sensitivity": 1.5,
    "restitution_coeff": 0.85,
    "display_brightness": 255
  },
  "control": {
    "reset_game": false,
    "force_pause": false
  }
}
```

5. CORE ALGORITHMS SPECIFICATION

A. 2D Physics Engine

- **Input Mapping:** Read linear accelerations (A_x , A_y) from the QMI8658 IMU. Transform raw values into screen-space directional force vectors through a low-pass complementary filter to eliminate high-frequency hand tremors.
- **Numerical Integration:** Apply explicit Euler integration at a fixed time-step (Δt):
$$\vec{v}_{\text{new}} = \vec{v}_{\text{old}} + (\vec{A}_{\text{mapped}} \cdot \Delta t)$$
$$\vec{p}_{\text{new}} = \vec{p}_{\text{old}} + (\vec{v}_{\text{new}} \cdot \Delta t)$$
- **Collision Response (Open Arena Reflection):** Implement axis-aligned bounding box (AABB) or circle-vs-line boundaries. When a display border violation occurs, the respective velocity vector component is inverted and scaled by a coefficient of restitution (e):
$$v_x = -v_x \cdot e$$

(On Vertical Wall Impact)

B. Maze Generation

- **Algorithm:** Randomized Depth-First Search (Recursive Backtracker) operating on a discrete mathematical grid mapped to the ST7789V2 LCD layout (e.g., 16×16 cell matrix).
- **Process Flow:** 1. Initialize all grid spaces as unvisited with walls intact on all four faces. 2. Choose a random starting cell, mark it as visited, and push it onto a tracking stack. 3. While the stack is not empty: pop a cell, check for unvisited neighbors. If found, pick one at random, knock down the shared wall between them, mark the new cell as visited, and push both cells back onto the stack.
- **Physics Interaction:** The 2D physics engine reads the generated cell wall array to execute static line-segment coordinate checks against the ball's radius vector to deny passage.

Circle-to-Grid-Maze Collision Detection

Prompt:

I am building a 2D tilt-controlled game called "BiteBound" for an ESP32-S3 paired with an ST7789 display (240x240px). The gameplay consists of a ball roll navigating a custom maze grid. We represent our board as a 1D uint8_t flat array of size width * height, where 0 represents a free corridor and 1 represents a wall.

I need a high-performance C++ class called MazeCollider that checks collisions for a circular PhysicsBody (holding x, y, and radius). To keep frame rates high on the microcontroller, the check should only scan wall cells within the body's bounding box instead of checking the whole board. It should return a Contact struct containing:

- hit (bool)
- penetration (float, maximum intersection depth)
- normal (Vec2, indicating the direction to push the body out of the wall)

Could you write the execution method for this collider? Keep the physics-to-grid mapping simple (1x1 units per cell) and optimize for the ESP32's floating-point unit.

Answer:

Algorithm: Bounding-Box Grid-Circle Collision Detection

```
function queryCollision(body):
```

```
  r = body.radius
```

1. BOUNDING BOX SPATIAL PRUNING

Limit grid checks strictly to tiles overlapping the circle's bounding box

```
x0 = floor(body.x - r)
```

```
x1 = ceil(body.x + r)
```

```
y0 = floor(body.y - r)
```

```
y1 = ceil(body.y + r)
```

Clamp coordinates to remain within grid boundaries

```
x0 = max(0, x0)
```

```
y0 = max(0, y0)
```

```
x1 = min(grid_width - 1, x1)
```

```
y1 = min(grid_height - 1, y1)
```

```
maxPenetration = 0.0
```

```
collisionNormal = [0.0, -1.0] Default fallback normal
```

```
hit = false
```

2. TILE EVALUATION

```
for py from y0 to y1:
```

```
  for px from x0 to x1:
```

```
    if board[px, py] is not WALL:
```

```
continue Skip empty corridors
```

3. CLOSEST POINT PROJECTION (AABB Clamping)

Clamp circle center onto the 1x1 tile bounds

```
nx = clamp(body.x, px, px + 1)
```

```
ny = clamp(body.y, py, py + 1)
```

4. METRIC COMPUTATIONS

```
dx = body.x - nx
```

```
dy = body.y - ny
```

```
distanceSquared = dx*dx + dy*dy
```

5. NARROW-PHASE SATISFIABILITY TEST

```
if distanceSquared < r * r:
```

```
    distance = sqrt(distanceSquared)
```

```
    penetration = r - distance
```

Track only the deepest cell penetration for single resolver

```
if penetration > maxPenetration:
```

```
    maxPenetration = penetration
```

```
hit = true
```

```
if distance > 1e-4:
```

```
    collisionNormal = [dx / distance, dy / distance]
```

```
else:
```

```
    collisionNormal = [0.0, -1.0] Perfect overlap fallback
```

```
if hit:
```

```
    return Contact(hit=true, penetration=maxPenetration, normal=collisionNormal)
```

```
else:
```

```
    return Contact(hit=false, penetration=0.0, normal=[0.0, 0.0])
```

Collision Logic & Structural Breakdown

The algorithm implements a **static Circle-vs-AABB (Axis-Aligned Bounding Box)** collision pipeline specialized for a uniform 2D grid:

- AABB Spatial Hashing/Narrow-phase Pruning:** Instead of checking the ball against all pixels in the entire maze, it determines the outer limits of the ball in grid coordinates (x_0 , x_1 , y_0 , y_1) using local flooring and ceiling operations. It limits evaluation to a maximum of a 3×3 or 4×4 grid area around the body, bringing the complexity down from $O(W \times H)$ to $O(R^2)$ (where R is the body's radius relative to cell size).
- Grid-cell Closest Point Projection:** For each colliding grid cell, it calculates the closest point (nx , ny) inside that specific 1×1 pixel block to the circle's center ($body.x$, $body.y$). This is achieved by clamping the circle center coordinates to the cell's physical bounding boundaries.

3. ****Narrow-Phase Separation Test:**** It computes the vector from the closest cell point to the circle center. If its magnitude is less than the radius, there is a physical overlap.
4. ****Local Deepest Penetration Filter:**** Rather than accumulating all overlapping impulses, the loop selectively records only the cell that yields the deepest penetration depth (maxPen). The contact normal is configured to point directly from that closest point to the center, indicating the shortest path vector required to push the ball out of the wall.

Typical Misconceptions & Vulnerabilities

When developers utilize or adapt this code, they often encounter several physical and mathematical misconceptions:

- ****The Single-Deepest-Penetration Fallacy (Corner Stuckness):****

By only tracking the single deepest penetration, the algorithm ignores multi-cell overlaps. If the ball rolls into an internal corner (colliding with both a bottom wall and a right wall), only the deeper wall is resolved. In the next frame, the physics engine pushes the ball directly sideways into the secondary wall block, leading to visual stuttering, deceleration, or the player getting stuck. A full impulse solver should accumulate response forces from all intersecting cells rather than selecting just one.

- ****Perfect Overlap Singularity (Zero-Distance Division):****

If the ball's center lands exactly on a cell border or perfectly inside a wall cell, the distance $d \approx 0$. At this point, normal calculation (dx/d , dy/d) becomes undefined or undergoes division by zero. The fallback normal `Vec2(0.0f, -1.0f)` is arbitrarily hardcoded. If the actual exit path was upwards, this fallback pushes the ball deeper into the wall, causing it to clip through.

- ****The Tunneling Vulnerability (High-Speed Clip):****

This algorithm performs static intersection tests. If the ball is moving at high speeds (where its displacement per frame exceeds its radius), it can jump over wall pixels completely between frames. To resolve this, continuous collision detection (CCD) or sub-stepping within the physics loop is needed.

- ****Uniform Scale Assumption:****

This engine assumes a hardcoded scale where 1 cell = 1 unit of physical coordinate. If the rendering or physical world scales up (e.g., each tile is 16times16 display pixels), this method will break unless coordinates are transformed into local grid space before the bounding box calculation.

Concurrency and FreeRTOS

Prompt:

I am developing an embedded 2D game on an ESP32-S3. The application has two main components with vastly different timing requirements: a high-frequency game

loop running at circa 50Hz (handling low-pass filtered IMU input, physics sub-stepping, collision resolution, and display rendering) and a low-frequency network stack running at circa 2Hz (handling blocking MQTT connections, TLS handshakes, and telemetry JSON serialization).

Running these sequentially stalls the frame rate whenever the network blocks.

Since the ESP32-S3 is a dual-core SoC, I want to use a way to separate these workloads across Core 0 and Core 1.

Can you provide a high-level architectural pattern using a library or a possible solution that is state of the art and industry standard?

Also compare possible solutions. Provide a problem-agnostic explanation and boilerplate code demonstrating how to set this up in C++ with Arduino on an ESP32-S3.

Answer:

The industry-standard, state-of-the-art pattern for this architecture on the ESP32-S3 is **Dual-Core Task Pinning with Thread-Safe Inter-Process Communication (IPC)** using the native FreeRTOS SDK embedded in the Arduino core.

Architectural Pattern: Asymmetric Core Isolation

The ESP32-S3 features two Xtensa LX7 cores. By default, the ESP32 Wi-Fi and Bluetooth stacks run on **Core 0**. Arduino's `setup()` and `loop()` run on **Core 1**.

To prevent network blocking from stalling your 50Hz game loop, you should isolate your workloads across the cores:

- * **Core 1 (Game Core):** Dedicated to the high-frequency game loop, IMU sampling, physics, and rendering. It must never perform blocking I/O.
- * **Core 0 (Network Core):** Handles the background Wi-Fi stack, TLS handshakes, MQTT processing, and serialization. It can block safely without affecting frame rates.

For IPC, **FreeRTOS Queues** are the standard choice for decoupling telemetry data, while **Mutexes** or **Double Buffering** are used if the network core needs to read a persistent snapshot of a large game state.

Comparison of Architectural Solutions

Solution Pattern	Pros	Cons	Best Used For
FreeRTOS Tasks + Queues	(Recommended)	Deeply integrated into ESP32; thread-safe; zero lock contention; non-blocking for the sender.	Copy-by-

```
value overhead for large structures (can be mitigated by passing pointers).  
| Passing discrete telemetry updates or event packets from Core 1 to Core 0.  
|  
| **Shared Memory + Mutexes** | Low memory overhead; ideal for reading large,  
persistent state blocks. | Introduces risk of priority inversion or frame  
drops if Core 0 holds the lock during a TLS write. | Sharing a persistent,  
global game state configuration. |  
| **Single-Core Cooperative (Async/State Machines)** | Minimal memory footprint;  
no multi-core concurrency bugs. | TLS handshakes and cryptographic  
calculations will *always* violate your 20ms (50Hz) frame budget. | Low-  
frequency systems without heavy cryptographic/blocking I/O. |
```

C++ Arduino Boilerplate Code

Here is a problem-agnostic template demonstrating how to pin tasks to separate
cores and safely stream telemetry data from your 50Hz game loop to a 2Hz
network stack.

```
# include <Arduino.h>  
  
// Structure for cross-core telemetry data sharing  
struct TelemetryData {  
    uint32_t frameCount;  
    float imuX;  
    float imuY;  
    unsigned long timestamp;  
};  
  
// FreeRTOS IPC Handle  
QueueHandle_t telemetryQueue;  
const int QUEUE_LENGTH = 5;  
  
// Task handle for the network loop  
TaskHandle_t NetworkTaskHandle = NULL;  
  
// Forward declaration of the Core 0 Network Task  
void networkTaskLoop(void *pvParameters);  
  
void setup() {  
    xTaskCreatePinnedToCore(  
        networkTaskLoop,    // Function to implement the task  
        "NetworkTask",      // Name of the task  
        8192,               // Stack size in words  
        NULL,               // Task input parameter  
        1,                  // Priority of the task  
        &NetworkTaskHandle, // Task handle
```

```
        0                // Core ID (Core 0 handles Wi-Fi/Network)
    );
}

// Network loop running exclusively on Core 0
void networkTaskLoop(void *pvParameters) {
    TelemetryData receivedData;

    // Local timing variables for managing the 2Hz cadence
    const TickType_t xDelay = pdMS_TO_TICKS(500); // 500ms = 2Hz

    for (;;) {
        // Blocks until a packet arrives or times out after 500ms
        if (xQueueReceive(telemetryQueue, &receivedData, xDelay) == pdTRUE) {

            Serial.printf("[Core %d] Processing Telemetry - Frame: %d, IMU_X: %.2
                f\n",
                        xPortGetCoreID(), receivedData.frameCount, receivedData.
                            imuX);

            // 1. Handle JSON Serialization
            // 2. Manage Blocking TLS / MQTT Publish operations
            // Simulating a heavy network stall (e.g., TLS Handshake or TCP
                Retransmit)
            // Core 1 will remain entirely unaffected by this delay
            delay(150);

        } else {
            // Keep-alive or heartbeat logic when no new telemetry is generated
            // e.g., client.loop() for MQTT maintaining connection state
        }
    }
}
```

Docstring Generation

To enhance code readability and accessibility, comments and docstrings were written for functions and classes in the codebase. The docstring writing was partly assisted by AI.

Prompt:

Generate a docstring for the following functions and classes. The docstring should be in the already used style format and should include a brief description of the function/class, its parameters, and its return value. The docstring should be concise and informative, providing enough information for a developer to understand the purpose and usage of the function/class without having to read the implementation details.

If needed, apply professional comments to clarify substeps in the function/class implementation.

Context:

Current file (cpp/h), optionally README.md text snippets, and any other relevant information can be used to generate the docstring.

Answer:

Generated docstrings.

Graphics Library Selection

Prompt:

I am developing an embedded 2D game on an ESP32-S3. The application has two main components with vastly different timing requirements: a high-frequency game loop running at circa 50Hz (handling low-pass filtered IMU input, physics sub-stepping, collision resolution, and display rendering) and a low-frequency network stack running at circa 2Hz (handling blocking MQTT connections, TLS handshakes, and telemetry JSON serialization).

I need to design a GraphicsManager class for the display.

Because of the tight 20ms frame budget on Core 1, full screen updates will stall the game. I need a rendering strategy that supports fast, partial updates to erase and redraw only changing elements (the ball, scores, and cookie collection states), plus a custom brand loading screen.

Which graphics library should I use for this type of low-level, high-frequency rendering on the ESP32: The lightweight GFX library or something more complex like LVGL? Please give a recommendation based on performance, explain its basic rendering usage, and provide a short, problem-agnostic example showing how to initialize the library and draw basic shapes.

Answer:

Recommendation: Arduino_GFX_Library vs. LVGL

For a 50Hz custom physics game loop running on a tight 20ms budget, Arduino_GFX_Library is highly recommended over LVGL (Light and Versatile Graphics Library).

- LVGL is designed for object-oriented, widget-based User Interfaces (buttons, sliders, animations). It introduces a virtual DOM-like layout tree and substantial memory/CPU overhead, making it difficult to optimize for raw, frame-by-frame game updates.
- Arduino_GFX_Library provides low-level, direct hardware access to pixel manipulation and fast primitive rendering. It is lightweight, allows you to implement manual dirty-rectangle tracking easily, and minimizes driver-level overhead, which is exactly what your Game Core requires.

Basic Graphics Usage Explanation

Graphics libraries interact with the display via a communications bus (SPI, I2S, etc.) and a specific panel driver. Once initialized, rendering generally follows three core concepts:

1. Canvas/Frame Buffers: Drawing to an off-screen buffer in RAM prevents screen flickering.
2. Primitives: Utilizing built-in hardware-accelerated methods for common shapes (lines, circles, rectangles).
3. Partial Redrawing (Dirty Tracking): Instead of clearing the entire screen, you selectively overwrite the old bounding box of an object with background color, then draw the object at its new coordinates.

GFX Library Usage Example

Here is a streamlined, clean example of how to initialize `Arduino_GFX` and perform basic, frame-controlled primitive drawing without application-specific logic.

```
# include <Arduino_GFX_Library.h>

// 1. Initialize the display data bus and driver panel (pins from config)
Arduino_DataBus *bus = new Arduino_ESP32SPI(DC_PIN, CS_PIN, SCK_PIN, MOSI_PIN,
      MISO_PIN);
Arduino_GFX*gfx = new Arduino_ST7789(bus, RST_PIN, ROTATION);

void setup() {
    // Start the display hardware connection
    if (!gfx->begin()) {
        Serial.println("Display initialization failed!");
        return;
    }

    // Fill background color initially
    gfx->fillScreen(BLACK);
}

void loop() {
    // Example primitive rendering sequence representing a single tick update

    // Erase previous state area (Mock Dirty Rect Optimization)
    int prevX = 50, prevY = 50, radius = 10;
    gfx->fillCircle(prevX, prevY, radius, BLACK);

    // Draw updated primitive state
    int newX = 55, newY = 50;
    gfx->fillCircle(newX, newY, radius, RED);
}
```

```
// Draw localized UI elements
gfx->setTextColor(WHITE);
gfx->setCursor(10, 10);
gfx->print("FPS: 50");

delay(20); // Maintain frame budget cadence
}
```

Logo Generation

Prompt:

A professional mobile app logo for a game titled "BiteBound", presented as a clean, centered circular icon on a transparent background. The design is playful, vibrant, and modern vector-style.

The central focus is a delicious, golden-brown chocolate chip cookie (using warm tones of cookie dough, honey yellow, and rich caramel). The cookie features glossy, melting dark chocolate chunks. Poised playfully on or next to the cookie is a vibrant, single glossy red berry character acting as the player element.

The game title text "BiteBound" can but doesn't have to be integrated beautifully into the circular composition using a bold, friendly, and readable typography in a warm milk-cream color.

Style: Clean 2D vector art, smooth gradients, crisp edges, suitable for an App Store icon.

Color Palette: Golden dough, honey yellow, rich caramel, dark cocoa brown, vibrant berry red, and warm milk cream.

Exclusions: No realistic photographic elements, no messy 3D renders, no complex backgrounds.

Answer:

Image

Random DFS Maze Generation Algorithm

Prompt:

Act as a computer science expert. For an embedded IoT game ("BiteBound"), I need to design a maze generation algorithm for a grid-mapped display space (conceptually 240x280 with variable wall thickness).

Provide an architectural breakdown of a Randomized Depth-First Search (DFS) algorithm with backtracking. Your response must include:

- Definition and core concepts of Randomized DFS in maze generation.

- A step-by-step description of cell visitation and backtracking mechanics.
- Structured, language-agnostic pseudo-code of the generation loop, stack operations, and random neighbor selection.

STRICT REQUIREMENT: Do not generate any functional source code (e.g., C++, Python). Provide only high-level logical pseudo-code and analysis. Keep your answer concise, technical, and focused on algorithmic design principles.

Answer:

BITEBOUND: RANDOMIZED DFS MAZE GENERATION ARCHITECTURE

1. CORE CONCEPTS & GRID MAPPING

Randomized Depth-First Search (DFS) generates a "perfect maze" (one unique path between any two points, no loops) by computing a spanning tree over a logical grid graph.

To adapt this to a 240x280 pixel space:

- Abstract the pixel canvas into a logical grid where:
- Total Unit Size (U) = Cell Width + Wall Thickness.
- Columns = $240 / U$, Rows = $280 / U$.

Memory Optimization: In embedded IoT systems, recursion risks call-stack overflow. Use an iterative approach with an explicit array-based stack in static RAM.

Cell State: Track visitation and wall states (North, South, East, West) using bitmasks packed into a single byte per cell.

2. CELL VISITATION & BACKTRACKING MECHANICS

The algorithm carves a path through solid walls using a random walk, backtracking when it encounters a dead end.

Step-by-Step Execution:

- Initialize: Mark all cells as unvisited with all walls intact. Pick a starting cell as `current_cell` and mark it visited.
- Evaluate: Check the four orthogonal neighbors of `current_cell`. Filter out out-of-bounds or already visited cells.
- Advance: If unvisited neighbors exist, select one at random. Push `current_cell` onto the backtrack stack. Break the wall between `current_cell` and the chosen neighbor. Move to the neighbor and mark it visited.
- Backtrack: If no unvisited neighbors exist, pop a cell from the backtrack stack and make it the `current_cell`.
- Terminate: Repeat steps 2-4 until the backtrack stack is empty.

3. LOGICAL PSEUDO-CODE

```
FUNCTION GenerateMaze(rows, cols):
Create 2D array 'grid' of size [rows][cols] (all visited = false, all walls
    intact)
Create array-based 'stack'
current_cell = [0, 0]
mark current_cell as visited

LOOP WHILE true:
    neighbors = GetUnvisitedNeighbors(current_cell, grid, rows, cols)

    IF neighbors is not empty THEN
        next_cell = RandomSelect(neighbors)
        PUSH current_cell TO stack
        RemoveWallsBetween(current_cell, next_cell, grid)
        current_cell = next_cell
        mark current_cell as visited
    ELSE IF stack is not empty THEN
        current_cell = POP from stack
    ELSE
        BREAK LOOP (Maze complete)
    END IF
END LOOP
END FUNCTION

FUNCTION GetUnvisitedNeighbors(cell, grid, rows, cols):
unvisited = empty list
directions = [[-1,0], [1,0], [0,-1], [0,1]] // North, South, West, East

FOR EACH dir IN directions:
    check_row = cell.row + dir[0]
    check_col = cell.col + dir[1]

    IF check_row within bounds AND check_col within bounds THEN
        IF grid[check_row][check_col].visited is false THEN
            ADD [check_row, check_col] TO unvisited
        END IF
    END IF
END FOR
RETURN unvisited
END FUNCTION
```

4. EMBEDDED DESIGN TIP

For high performance on low-power displays, update the screen incrementally by drawing the "carved" paths during step 3, rather than redrawing the whole grid every loop cycle.

NodeRED Dashboard Library Decision

Prompt:

I am developing an embedded 2D game on an ESP32-S3. The application has two main components with vastly different timing requirements: a high-frequency game loop running at circa 50Hz (handling low-pass filtered IMU input, physics sub-stepping, collision resolution, and display rendering) and a low-frequency network stack running at circa 2Hz (handling blocking MQTT connections, TLS handshakes, and telemetry JSON serialization).

We need to establish our Node-RED dashboard architecture under the ``nodered/`` directory. The use case requires the dashboard to act as a master operator console that:

1. Displays real-time game telemetry (ball positioning, score states, cookie field configurations) updated every 0.5s.
2. Injects control payloads (START / STOP triggers and runtime variable modifications) into the ESP32 queue via MQTT.

What is the recommended library or architectural approach to build this interface within Node-RED to ensure long-term support, performance, and UI flexibility?

Answer:

The recommended approaches for building a Node-RED dashboard interface for your ESP32-S3 embedded 2D game are as follows:

- `flowfuse/node-red-dashboard` (Dashboard 2.0): This is the official, modern ecosystem standard for Node-RED dashboards. It is actively maintained and built on Vue 3 + Vuetify, providing a low-code configuration with code-hybrid templates. It supports native Vue reactivity, which can handle JSON streams instantly, making it suitable for your 0.5s telemetry updates. The modular ``ui-template`` allows for embedded CSS/Vue components, offering flexibility in UI customization.
- `node-red-contrib-uibuilder`: This is a long-standing community-driven alternative that is framework agnostic, allowing you to use Vanilla JS, React, or Svelte. It is code-first, meaning you manage front-end build pipelines yourself. It uses dedicated WebSockets for high-performance data streaming and provides absolute freedom in UI design, but requires more effort to set up and maintain.

Framework Evaluation for Your Operator Console

Metric / Feature	<code>`@flowfuse/node-red-dashboard`</code> (Dashboard 2.0)	<code>`node-red-contrib-uibuilder`</code>
---	---	---
LTS Status	**Active** (The official, modern ecosystem standard)	**Active** (Long-standing community-driven alternative)

```
| **Core Architecture** | Vue 3 + Vuetify | Framework Agnostic (Vanilla JS,  
    React, Svelte) |  
| **Development Model** | Low-Code configuration with code-hybrid templates |  
    Code-First (You manage front-end build pipelines) |  
| **2Hz Telemetry Sync** | Native Vue reactivity handles JSON streams instantly  
    | Uses dedicated WebSockets; highly performant |  
| **UI Customization** | Modular `ui-template` allows embedded CSS/Vue  
    components | Absolute freedom, but requires writing all UI scaffolding |
```

****Verdict:**** Choose ****Dashboard 2.0****. It provides out-of-the-box UI controls (sliders, buttons, switches) for your runtime control payloads, while offering the flexibility of custom Canvas/SVG nodes for rendering real-time game telemetry.

To support the BiteBound telemetry and control system, you should utilize the Node-RED UI Builder Dashboard (node-red-contrib-uibuilder) rather than the standard Node-RED dashboard nodes.

The primary driver for this architectural decision is that the original, traditional "node-red-dashboard" library is officially deprecated and no longer maintained for modern Node-RED versions. Continuing with the legacy library introduces security liabilities and lack of support for modern Node.js runtimes.

By selecting UI Builder for this use case, you gain several advantages:

- Data-Stream Performance: UI Builder decouples the frontend from the Node-RED backend using optimized WebSockets. This ensures that the 0.5s telemetry snapshots do not cause lag or choke the backend flow orchestrations.
- Bidirectional Control: It cleanly maps frontend events (clicking START/STOP or sliding parameter configurations) to Node-RED messages, which can immediately be routed to your Core 0 MQTT publish nodes.
- Frontend Freedom: You can build a modern, responsive UI using vanilla HTML/JS or lightweight frameworks (like Vue or custom CSS grids) to clearly map out the game state without being restricted by the rigid grid of the deprecated library.

Android to Node-RED Migration Examples

In order to guarantee as few as possible changes to the existing frontend styling and logic, code examples and patterns were evaluated and selected to be as close as possible to the original Android codebase.

Prompt:

I need code examples on migrating an IoT companion app's frontend from Android (Kotlin/Jetpack Compose) to a Node-RED UI Builder web interface (HTML/CSS/Vanilla JS).

Using the provided context (parts of our Android codebase and the current project README), please present code equivalents for the core styles in CSS, UI components in HTML, and internal logic in Vanilla JS.

The examples are not expected to be complete, but should illustrate the general patterns and key differences to the implementations in the Android codebase. Keep the code examples concise and focused on the core concepts, use helpful comments and reduce complexity where possible.

Context:

A few selected single files from the Android Source Code, and the README describing the current architecture and design patterns.

Answer:

The code migrations from Android (Kotlin/Jetpack Compose) to Node-RED UI Builder (HTML/CSS/Vanilla JS).

Android to Node-RED Migration Guide

Prompt:

I need a technical guide on migrating an IoT companion app's frontend from Android (Kotlin/Jetpack Compose) to a Node-RED UI Builder web interface (HTML/CSS/Vanilla JS).

Using the provided context (parts of our Android codebase and the current project README), please outline:

1. A mapping of Android concepts to Web/Node-RED equivalents.
2. Architectural guidelines for structuring the HTML, CSS, and JS files modularly.
3. Simple code templates demonstrating how to parse incoming telemetry JSON cleanly and dispatch structured commands back via `uibuilder.send()` over WebSockets.

Focus on the general patterns rather than our game's business logic, and explain the key differences in state management between Jetpack Compose and modern Vanilla JavaScript.

Context:

Relevant parts of the Android Source Code, including basic styling, constants, modelview, JSON commands, MQTT handling, and the README describing the current architecture and design patterns.

Answer:

Migrating an IoT companion **interface** from a **native** Android application to a Node-RED UI Builder web dashboard involves moving from a statically typed, declarative JVM environment to a dynamic, event-driven web runtime.

1. CONCEPT MAPPING

+-----+-----+		
Android (JVM / Kotlin)	Web / UI Builder Equivalent	
+-----+-----+		
SharedPreferences	LocalStorage API	
Jetpack Compose State	Stateful JS Object + DOM Binder	
MqttAsyncClient	uibuilder WebSocket connection	
Coroutine Delay/Timeout	setTimeout() / clearTimeout()	
+-----+-----+		

2. ARCHITECTURAL STRUCTURING

UI Builder uses a modular directory structure under `'src/'`. To mirror Android's separation of concerns, split the codebase as follows:

- `index.html`: Declarative markup structure (the layouts).
- `index.css`: Visual properties and CSS Custom Properties (the theme).
- `index.js`: Entry point coordinating events and state changes.
- `app/types.js`: Holds factories and default configurations for fallback handling.
- `app/command.js`: Assembles and dispatches commands over WebSockets.
- `app/render.js`: Dynamic rendering engine updating the DOM tree.

3. BASIC GENERIC PATTERNS

A. Saving and Loading Connection Profiles (LocalStorage):

```
// Saving
const credentials = { host: "localhost", port: 1883 };
localStorage.setItem('my_app_creds', JSON.stringify(credentials));

// Loading
const stored = localStorage.getItem('my_app_creds');
const myCreds = stored ? JSON.parse(stored) : { host: "default", port: 1883 };
```

B. Parsing Telemetry securely with Fallbacks:

```
function parseTelemetry(rawPayload) {
  let json = {};
  try {
    json = (typeof rawPayload === 'string') ? JSON.parse(rawPayload) :
    rawPayload;
  } catch (e) {
    console.error("Payload error", e);
  }
}
```

```
    return {
      status: json.status || "unknown",
      value: typeof json.value === 'number' ? json.value : 0.0
    };
  }
}
```

C. Dispatching Outbound Commands via UI Builder:

```
function sendCommand(uibuilder, action, parameters) {
  const payload = {
    command: action,
    timestamp: new Date().toISOString(),
    payload: parameters
  };
  uibuilder.send({
    payload: payload,
    topic: "my_app/command"
  });
}
```

4. STATE MANAGEMENT DIFFERENCES

- Jetpack Compose uses a compiler plugin to automatically track reads/writes to mutable states (e.g. `mutableStateOf`) and trigger micro-recompositions of corresponding UI trees.
- In Vanilla JS, state updates do not automatically re-render elements. To manage state, store parameters inside a plain JavaScript object and call explicit `render/update` functions whenever a mutating event occurs.

Physics Engine Architecture Prompt

Prompt:

I am building a physics engine for an ESP32-S3 game (50Hz loop). I need to move a ball based on IMU tilt. What is the best architectural approach? What sensor data is needed, and what is the optimal way to calculate physics stably without jitter? I need high adjustability.

Answer:

For a stable, adjustable 50Hz ESP32 game loop, follow these steps:

1. Sensors: Read raw IMU accelerometer pitch and roll.
2. Filtering: Apply an Exponential Moving Average (EMA) to smooth noise, plus a deadzone to prevent resting drift.
3. Physics Math: Use Semi-Implicit Euler integration. Update velocity from acceleration, then position from the new velocity. It is highly stable and computationally efficient for MCUs.
4. Tuning: Isolate sensitivity settings. Use separate parameters for input scaling, friction, and restitution.

Graphics Usage: Keep rendering entirely decoupled. The physics engine updates the spatial state on Core 1, and the graphics manager simply clears the dirty rectangle and redraws the shape at the new coordinates.

PlantUML Basic Template

Prompt:

I am developing an embedded 2D game on an ESP32-S3. The application has two main components with vastly different timing requirements: a high-frequency game loop running at circa 50Hz (handling low-pass filtered IMU input, physics sub-stepping, collision resolution, and display rendering) and a low-frequency network stack running at circa 2Hz (handling blocking MQTT connections, TLS handshakes, and telemetry JSON serialization).
I need a high-level architectural skeleton for an ESP32-based IoT game.

Please provide a basic object-oriented design and PlantUML outline showing the key components (Main, Network, Graphics, Physics, Sensors) and how they should safely interact or share state via FreeRTOS. Keep it foundational so we can refine it further later on. No complex game logic needed yet, just the high-level class layout and structural breakdown.

Answer:

Here is a foundational architecture blueprint designed to split workloads cleanly across both ESP32 cores, utilizing a synchronized shared state. This initial layout establishes the core boundaries before introducing complex interface abstractions.

Core Architecture Breakdown

1. Core 1 (Game Thread): Drives the deterministic execution loop. It polls sensors, steps the physics simulation, and flushes the buffer to the display.
2. Core 0 (Network Thread): Operates asynchronously, dealing with unpredictable I/O latencies from WiFi and MQTT brokers, parsing incoming commands into message queues.
3. Shared Data: A thread-safe data bucket protected by a FreeRTOS Mutex Semaphore to prevent race conditions during telemetry reads and command writes.

Conceptual Graphics Management

In a dual-core setup, the Graphics Manager on Core 1 typically operates on a non-blocking draw cycle synchronized with the physics timestep (e.g., 50Hz / 20 ms). Instead of clearing the entire screen (which induces heavy SPI overhead and lag), it should follow a basic dirty-rectangles or differential drawing pattern:

- Erase previous frame's moving entity positions using background-colored primitives.
- Draw new entity positions.
- Render static UI overlays only on state changes.

Initial PlantUML Skeleton

@startuml BasicGameArchitecture
top to bottom direction

```
package "Core 0 (Networking & Async I/O)" {
    class WifiManager {
        + connect() : bool
        + isConnected() : bool
    }
    class MqttManager {
        + init() : void
        + publishTelemetry(payload : String) : bool
        + handleIncomingCommand(topic : char*, payload : byte*) : void
    }
}

package "Shared Space (IPC)" {
    struct SharedSystemState {
        + ballX : float
        + ballY : float
        + score : int
        + systemStatus : int
    }
    class MutexWrapper {
        + lock() : void
        + unlock() : void
    }
}

package "Core 1 (Game Engine & Peripherals)" {
    class MainApp {
        + setup() : void
        + gameLoop() : void
    }
    class SensorManager {
        + readIMU() : void
    }
    class PhysicsEngine {
        + updatePosition(ax : float, ay : float) : void
    }
}
```

```
class GraphicsManager {
    + initDisplay() : void
    + renderFrame(state : SharedSystemState) : void
}

' Interactions
MainApp --> SensorManager : "Polls data"
MainApp --> PhysicsEngine : "Steps simulation"
MainApp --> GraphicsManager : "Triggers redraw"
MainApp ..> MutexWrapper : "Protects writes"
MainApp ..> SharedSystemState : "Updates state"

MqttManager ..> WifiManager : "Depends on"
MqttManager ..> MutexWrapper : "Protects reads"
MqttManager ..> SharedSystemState : "Reads for telemetry"

@enduml
```

README Generation

To enhance code readability and accessibility, README files were written for the project. The README writing was partly assisted by AI.

Prompt:

I am developing an embedded 2D game on an ESP32-S3. The application has two main components with vastly different timing requirements: a high-frequency game loop running at circa 50Hz (handling low-pass filtered IMU input, physics sub-stepping, collision resolution, and display rendering) and a low-frequency network stack running at circa 2Hz (handling blocking MQTT connections, TLS handshakes, and telemetry JSON serialization).

Use the provided context to append to, enhance and update the README.md file for the project with the most recent changes.

Use headings, subheadings, bullet points, inline code, notes, and other formatting to make the README.md file more readable and keeping it professional.

Do not change the provided README structure, but enhance it with the most recent changes and improvements.

Summarize the code changes and provide a brief overview about the usage.

Context:

Current file (cpp/h), README.md text snippets, and any other relevant information can be used to generate the docstring.

Answer:

Generated README.md content with the most recent changes and improvements.

Unit Test Generation

To ensure code reliability and maintainability, unit tests were developed for the core hardware components. The unit test writing was partly assisted by AI.

Prompt:

Generate comprehensive unit tests for the following functions and classes. The tests should cover normal cases, edge cases, and error conditions. Each test should be independent, focused on a single behavior, and include clear assertions. Use descriptive test names that indicate what is being tested. Include setup and teardown methods where necessary to ensure test isolation and reproducibility.

Context:

Current file (cpp/h), optionally README.md text snippets, and any other relevant information can be used to generate the unit tests.

Answer:

Generated unit tests.

B ESP32 Code

The main control logic for the ESP32 microcontroller, including the core game loop, hardware abstraction, and network communication, is presented in this section.

As the requirements state that the ESP32 code should be part of the appendix, the code listings are included here for reference.

B.1 esp32.ino

Path: esp32/esp32.ino

```
#include "src/network/time/TimeManager.h"
#include "src/network/wifi-connection/WifiManager.h"
#include "src/network/mqtt/MqttManager.h"
#include "src/network/json-builder/JsonBuilder.h"
#include "src/network/json-parser/CommandParser.h"
#include "src/network/shared/SharedStateData.h"
#include "src/network/shared/MutexLock.h"
#include "src/sensors/SensorManager.h"
#include "src/graphics/GraphicsManager.h"
#include "src/engine/GameEngine.h"
#include "config.h"
```

```
#include <AUnit.h>
#include <ArduinoJson.h>
#include <Arduino_GFX_Library.h>
#include <freertos/FreeRTOS.h>
#include <freertos/semphr.h>
#include <freertos/queue.h>

// Set to 1 to use mock sensor data instead of real hardware
#ifndef MOCK_SENSORS
#define MOCK_SENSORS 0
#endif

// Set to 1 to run AUnit tests; set 0 for main functionality.
// Can be overridden at compile time via -DRUN_TESTS=1 (used by CI).
#ifndef RUN_TESTS
#define RUN_TESTS 0
#endif

// Threading Synchronizations
SemaphoreHandle_t stateMutex = nullptr;
QueueHandle_t commandQueue = nullptr;
SharedStateData sharedState;

// Global Graphics Setup
Arduino_DataBus *bus = new Arduino_ESP32SPI(pin_lcd_dc, pin_lcd_cs, pin_lcd_sck,
    pin_lcd_mosi);
Arduino_GFX *gfx = new Arduino_ST7789(bus, pin_lcd_rst /* RST */, 0 /* rotation
    */,
    true /* IPS */, display_width, display_height, 0, 20, 0, 0);
GraphicsManager graphicsManager(display_width, display_height);

// Play space calculations (HUD occupies the top ui_header_height pixels)
const int play_width = display_width;
const int play_height = display_height - ui_header_height;

// Game orchestration: owns physics and both games, and drives the active one.
GameEngine gameEngine;

// FreeRTOS Task on Core 0 to isolate network operations.
void vNetworkTask(void *pvParameters);

// Helper function declarations
void processIncomingCommands();
void updateGameStep();

/**
 * @brief Main setup function for the MCU controller.
```

```
* This function initializes processing primitives, connectivity, and the game
  loop.
*/
void setup() {
  Serial.begin(115200);
  Serial.println("ESP32 BiteBound Hardware starting...");

  // Initialize threading primitives
  stateMutex = xSemaphoreCreateMutex();
  commandQueue = xQueueCreate(cmd_queue_length, cmd_queue_item_size);
  initSharedState(sharedState);

#ifdef RUN_TESTS
  // serial initialization delay for test output
  delay(3000);
  Serial.println("Starting AUnit tests...");
#else
  // Initialize Display and Graphics Manager
  if (!gfx->begin()) {
    Serial.println("Failed to initialize GFX display!");
  }
  pinMode(pin_lcd_bl, OUTPUT);
  digitalWrite(pin_lcd_bl, HIGH);
  graphicsManager.begin(gfx);

  // Connect to WiFi
  graphicsManager.drawLoadingScreen("Connecting WiFi...");
  wifiManager.connect();

  // Synchronize time (required for TLS certificate validation and timestamps).
  graphicsManager.drawLoadingScreen("Syncing Time...");
  timeManager.sync();

  // Seed standard library random engine with timing noise
  std::srand(micros());

  // Initialize sensors
  graphicsManager.drawLoadingScreen("Initializing Sensors...");
  sensorManager.begin();

  // Setup MQTT client
  graphicsManager.drawLoadingScreen("Starting MQTT...");
  mqttManager.begin();
  mqttManager.connect(); // initial connection attempt
  mqttManager.loop(); // once to verify connection

  // Initialize the game engine and start in idle mode.
```



```
graphicsManager.drawLoadingScreen("Creating Maze...");
gameEngine.begin(play_width, play_height);
gameEngine.chooseGameMode(default_game_id);

// Set running state to true to match main branch behavior (playable on boot).
{
    MutexLock lock(stateMutex);
    if (lock.isLocked()) {
        sharedState.gameId = default_game_id;
        sharedState.runningStatus = RunningStatus::RUNNING;
    }
}

// Create Network Task on Core 0 (isolated from logic/rendering)
xTaskCreatePinnedToCore(
    vNetworkTask,
    "NetworkTask",
    8192,
    NULL,
    1,
    NULL,
    core_network
);

graphicsManager.drawLoadingScreen("BiteBound is Ready!");
#endif
}

/**
 * @brief Main loop function for the MCU controller (running on Core 1).
 */
void loop() {
#ifdef RUN_TESTS
    // aunit::TestRunner::list(); // List all registered tests.
    aunit::TestRunner::setTimeout(30); // Set a timeout of 30 seconds for tests.
    aunit::TestRunner::run();
#else
    static TickType_t lastTickTime = xTaskGetTickCount();

    // 1. Core 1 Command Processing
    processIncomingCommands();

    // 2. Local Game Step & Rendering
    updateGameStep();

    // Preserve precision step constraints (e.g. 50Hz for smooth physics)
    vTaskDelayUntil(&lastTickTime, pdMS_TO_TICKS(game_tick_rate_ms));
#endif
}
```

```
}

/**
 * @brief Processes incoming command structures transmitted from the network
 *        task.
 */
void processIncomingCommands() {
    CommandMsg commandMsg;
    while (xQueueReceive(commandQueue, &commandMsg, 0) == pdPASS) {
        {
            MutexLock lock(stateMutex);
            if (lock.isLocked()) {
                if (commandMsg.type == CommandType::START) {
                    GameConfig config;
                    config.gameId = commandMsg.gameId;
                    config.targetCookies = commandMsg.cookiesCount;
                    // Don't show more cookies at once than are still needed to win, so a
                    // target of 1 shows exactly 1 cookie instead of the default 4.
                    config.visibleCookies = (commandMsg.cookiesCount <
                        default_max_visible_cookies)
                        ? (uint8_t)commandMsg.cookiesCount
                        : (uint8_t)default_max_visible_cookies;
                    config.wallThicknessPx = commandMsg.wallThicknessPx;
                    config.physics.sensitivity = commandMsg.imuSensitivity;
                    config.physics.restitution = commandMsg.bounceRestitution;
                    config.physics.emaAlpha = commandMsg.emaAlpha;
                    config.physics.deadzone = commandMsg.deadzoneThreshold;

                    gameEngine.applyConfig(config);
                    gameEngine.chooseGameMode(commandMsg.gameId);

                    sharedState.runningStatus = RunningStatus::RUNNING;
                    sharedState.gameId = commandMsg.gameId;
                    // Mirror the chosen target/wall into shared state so the telemetry (
                    // and
                    // with it the dashboard's cookie counter) reports the right values.
                    sharedState.cookiesCount = commandMsg.cookiesCount;
                    sharedState.wallThicknessPx = commandMsg.wallThicknessPx;
                    strncpy(sharedState.playerName, commandMsg.playerName, sizeof(
                        sharedState.playerName) - 1);
                }
                else if (commandMsg.type == CommandType::STOP) {
                    // Tell the engine to stop too, not just the shared flag. Otherwise the
                    // game step writes the running state back from the engine next frame.
                    gameEngine.stop();
                    sharedState.runningStatus = RunningStatus::IDLE;
                }
                else if (commandMsg.type == CommandType::RESUME) {
```

```
// Continue a paused game right where it left off (no rebuild). resume
()
// ignores it if there's no paused game, so a stray resume is harmless.
if (gameEngine.canResume()) {
    gameEngine.resume();
    sharedState.runningStatus = RunningStatus::RUNNING;
}
}
else if (commandMsg.type == CommandType::PARAM_CHANGE) {
    GameConfig config;
    config.physics.sensitivity = commandMsg.imuSensitivity;
    config.physics.restitution = commandMsg.bounceRestitution;
    config.physics.emaAlpha = commandMsg.emaAlpha;
    config.physics.deadzone = commandMsg.deadzoneThreshold;
    gameEngine.applyConfig(config);
}
}
}

// Debugging output for command processing is outside the critical lock
// section, as printing to Serial can be slow and blocking.
if (commandMsg.type == CommandType::START) {
    Serial.println("Command: Start game received.");
} else if (commandMsg.type == CommandType::STOP) {
    Serial.println("Command: Stop game received.");
} else if (commandMsg.type == CommandType::RESUME) {
    Serial.println("Command: Resume game received.");
} else if (commandMsg.type == CommandType::PARAM_CHANGE) {
    Serial.println("Command: Parameter Change received.");
} else {
    Serial.println("Command: Unknown command type received.");
}
}
}

/**
 * @brief Computes physics, game state transitions and rendering on Core 1.
 *
 * This function is called at a fixed rate (50Hz) to ensure smooth physics and
 * rendering.
 * It reads the latest sensor data, updates the game engine, renders the display
 * , and updates
 * the shared state for telemetry.
 */
void updateGameStep() {
    // 1. Sensors: Read sensor values (mock or real hardware)
    #if MOCK_SENSORS
        SensorData sensorData = sensorManager.readMock();
```

```
#else
    SensorData sensorData = sensorManager.read();
#endif

// Avoid unnecessary updating of the game engine if the game is not running.
bool isRunningStatus = false;
{
    MutexLock lock(stateMutex);
    if (lock.isLocked()) {
        isRunningStatus = (sharedState.runningStatus == RunningStatus::RUNNING)
        ;
    }
}

// 2. Time: Frame timing calculation
static uint32_t lastFrameMs = 0;
const uint32_t nowMs = millis();
if (lastFrameMs == 0) lastFrameMs = nowMs;
const float dt = (nowMs - lastFrameMs) / 1000.0f;
lastFrameMs = nowMs;

// 3. Physics: Advance the active game by one physics frame if running.
if (isRunningStatus) {
    // Coordinate system is rotated by 90 degrees.
    gameEngine.update(-sensorData.accelerometerY, sensorData.accelerometerX, dt)
    ;
}

// 4. Display: Render the current game state to the display.
if (gameEngine.consumeRedraw()) {
    graphicsManager.forceFullRedraw();
}
graphicsManager.update(
    gameEngine.board(),
    play_width,
    play_height,
    gameEngine.ball(),
    gameEngine.cookies().data(),
    (int)gameEngine.cookies().count(),
    gameEngine.state());

// 5. Telemetry: Update shared state for the network core to publish as
telemetry.
{
    MutexLock lock(stateMutex);
    if (lock.isLocked()) {
        const GameState &gs = gameEngine.state();
        const PhysicsBody &ball = gameEngine.ball();
```

```
sharedState.ballPosX = ball.x;
sharedState.ballPosY = ball.y;
sharedState.velocityX = ball.vx;
sharedState.velocityY = ball.vy;
sharedState.accX = 0; // Not currently calculated separately from tilt
    inputs
sharedState.accY = 0;

sharedState.cookiesCollected = gs.cookiesCollected;
sharedState.cookiesRemaining = gs.cookiesRemaining;
sharedState.currentRound = gs.currentRound;
sharedState.elapsedTimeSec = gs.elapsedTimeSec;
sharedState.gameId = gameEngine.activeGameId();
sharedState.runningStatus = gs.runningStatus;
}
}

// 6. Round complete: hold the "completed" state for a moment so the dashboard
// (and the on-device banner) can show it, then auto-start the next round
// (fresh maze / field, round + 1).
static uint32_t completedSinceMs = 0;
bool advanceRound = false;
{
    MutexLock lock(stateMutex);
    if (lock.isLocked()) {
        if (sharedState.runningStatus == RunningStatus::COMPLETED) {
            if (completedSinceMs == 0) {
                completedSinceMs = millis();
            } else if (millis() - completedSinceMs >= round_complete_hold_ms) {
                advanceRound = true;
            }
        } else {
            completedSinceMs = 0;
        }
    }
}
}
if (advanceRound) {
    // nextRound() rebuilds the level (new maze for Game 1) and sets RUNNING
    again.
    gameEngine.nextRound();
    completedSinceMs = 0;
    MutexLock lock(stateMutex);
    if (lock.isLocked()) {
        sharedState.runningStatus = RunningStatus::RUNNING;
        sharedState.currentRound = gameEngine.state().currentRound;
    }
}
```

```
}

/**
 * @brief Concurrent FreeRTOS network management task pinned to Core 0.
 * Handles MQTT connection, keeps the client alive, and publishes telemetry.
 */
void vNetworkTask(void *pvParameters) {
    TickType_t lastTelemetryTime = xTaskGetTickCount();

    // Retry MQTT if disconnected, but avoiding spamming the broker with
    // connection attempts. Use a cooldown interval.
    TickType_t lastMqttRetryTime = 0;
    const TickType_t mqttRetryInterval = pdMS_TO_TICKS(mqtt_retry_interval_ms);

    while (true) {
        // 1. Service MQTT loop and maintain connection status
        // Must be called frequently to handle keep-alives and incoming messages.
        if (!wifiManager.isConnected()){
            // Reconnect WiFi if lost
            Serial.println("NetworkTask: WiFi connection lost, attempting to reconnect
                ...");
            wifiManager.connect();
        }

        if (wifiManager.isConnected()) {
            mqttManager.loop();
        }

        // 2. Periodic telemetry publishing
        TickType_t currentTick = xTaskGetTickCount();

        // Reconnect MQTT if disconnected
        if (wifiManager.isConnected() && !mqttManager.isConnected()) {
            if (currentTick - lastMqttRetryTime >= mqttRetryInterval) {
                lastMqttRetryTime = currentTick;
                mqttManager.connect();
            }
        }

        if ((currentTick - lastTelemetryTime) >= pdMS_TO_TICKS(telemetry_rate_ms)) {
            lastTelemetryTime = currentTick;

            // Only publish if connected
            if (wifiManager.isConnected() && mqttManager.isConnected()) {
                SensorData sensorData = sensorManager.getLastData();
                TelemetryData telemetry;

                // Populate hardware information.
```

```
telemetry.client_id = device_id;
telemetry.hardware = device_hardware;
telemetry.firmware_version = device_firmware_version;
telemetry.uptime_ms = millis();
telemetry.wifi_ssid = wifiManager.getSSID();

// Populate raw sensor readings.
telemetry.accel_x = sensorData.accelerometerX;
telemetry.accel_y = sensorData.accelerometerY;
telemetry.accel_z = sensorData.accelerometerZ;
telemetry.gyro_x = sensorData.gyroscopeX;
telemetry.gyro_y = sensorData.gyroscopeY;
telemetry.gyro_z = sensorData.gyroscopeZ;
telemetry.battery_voltage = sensorData.batteryVoltage;
telemetry.button = sensorData.button;

// Thread-safe capture of current game progress and physics state.
{
    MutexLock lock(stateMutex);
    if (lock.isLocked()) {
        telemetry.game_id = sharedState.gameId;
        telemetry.player_name = sharedState.playerName;
        telemetry.target_cookies = sharedState.cookiesCount;
        telemetry.screen_width = display_width;
        telemetry.screen_height = display_height;
        telemetry.wall_thickness_px = sharedState.wallThicknessPx;

        String runningStatusStr = "idle";
        if (sharedState.runningStatus == RunningStatus::RUNNING) {
            runningStatusStr = "running";
        } else if (sharedState.runningStatus == RunningStatus::COMPLETED) {
            runningStatusStr = "completed";
        }
        telemetry.runningStatus = runningStatusStr;

        telemetry.cookies_collected = sharedState.cookiesCollected;
        telemetry.cookies_remaining = sharedState.cookiesRemaining;
        telemetry.current_round = sharedState.currentRound;
        telemetry.elapsed_time_sec = sharedState.elapsedTimeSec;

        telemetry.ball_pos_x = sharedState.ballPosX;
        telemetry.ball_pos_y = sharedState.ballPosY;
        telemetry.velocity_x = sharedState.velocityX;
        telemetry.velocity_y = sharedState.velocityY;
        telemetry.acc_x = sharedState.accX;
        telemetry.acc_y = sharedState.accY;
    }
}
```

```
        // Send telemetry JSON payload to MQTT broker
        String payload = buildTelemetryJson(telemetry);
        mqttManager.publish(mqtt_telemetry_topic, payload.c_str(), mqtt_retain);
    }
}

// Short yield to feed the IDLE task and watchdogs.
// vTaskDelay(): Delays the task for a specified number of ticks.
// pdMS_TO_TICKS(): Converts milliseconds to ticks.
vTaskDelay(pdMS_TO_TICKS(10));
}
}
```

B.2 Headers

The following listings contain all header files from the esp32/ project root and source tree. Note that wifi_mqtt_secrets_template.txt is included instead of the actual secrets file for security reasons.

config.h

Path: esp32/config.h

```
#ifndef CONFIG_H
#define CONFIG_H

#include <Arduino.h>

/**
 * Further configuration options for the ESP32.
 */

/** ----- Network settings ----- */
/** Device ID for identifying the ESP32 device on the network. */
const char *const device_id = "BiteBound-ESP32-S3-001";

/** Device hardware description. */
const char *const device_hardware = "Waveshare ESP32-S3 1.69inch";

/** Device firmware version. */
const char *const device_firmware_version = "1.0.0";

/** ----- MQTT settings ----- */
/** HiveMQ Cloud broker port. */
const int mqtt_port = 8883;

/** MQTT keep alive interval in seconds. */
```



```
const int mqtt_keep_alive_sec = 60;

/** MQTT topic for sending commands to the ESP32 device. */
const char *const mqtt_command_topic = "mauc2026/group_03/game/command";

/** MQTT topic for sending telemetry data from the ESP32 device. */
const char *const mqtt_telemetry_topic = "mauc2026/group_03/game/telemetry";

/** MQTT topic for sending test data from the ESP32 device. */
const char *const mqtt_test_topic = "mauc2026/group_03/test";

/** Quality of Service level for MQTT messages. */
const int mqtt_qos = 1;

/** Retain flag for MQTT messages. */
const bool mqtt_retain = false;

/** MQTT Retry interval in milliseconds. */
const int mqtt_retry_interval_ms = 2000;

/** MQTT Retry Attempts per connection setup. */
const int mqtt_retry_attempts = 2;

/** ----- Game settings ----- */

/** Game ID for the maze game. */
const int game_id_maze = 1;
/** Game ID for the plane game. */
const int game_id_plane = 2;
/** Default game id. */
const int default_game_id = game_id_maze;

/** Default game duration in seconds. */
// const int default_game_duration_sec = 120;

/** Default player name. */
const char *const default_player_name = "Cookie-Monster";

/** Default maximum number of cookies in the game. */
const int default_cookies_count = 10;

/** Default maximum number of visible cookies in the game. */
const int default_max_visible_cookies = 4;

/** How long the "completed" state is shown before the next round auto-starts [
    ms]. */
const uint32_t round_complete_hold_ms = 2000;
```

```
/** Default cookie radius in pixels. */
const float default_cookie_radius = 5.0f;

/** Default physics body (sphere/ball) radius in pixels. Only a fallback before
    a
    * level is built - the actual ball size is derived from the wall thickness
    below. */
const float default_physics_body_radius = 5.0f;

/** Ball diameter as a fraction of the wall thickness, so the ball always fits
    the
    * corridor no matter which wall thickness is chosen. radius = wall * ratio /
    2. */
const float ball_diameter_wall_ratio = 0.7f;

/** Maze cookie diameter as a fraction of the wall thickness, so cookies scale
    with
    * the corridor like the ball (a bit smaller so they stay distinguishable). */
const float cookie_diameter_wall_ratio = 0.6f;

/** Game 2 (open field) has no corridor to scale to, so its ball and cookies use
    * fixed sizes - picked to be clearly visible without dominating the field. */
const float default_flatland_ball_radius = 8.0f;
const float default_flatland_cookie_radius = 6.0f;

/** ----- Display settings ----- */
/** Display width in pixels. */
const int display_width = 240;

/** Display height in pixels. */
const int display_height = 280;

/** Default wall thickness in pixels. */
const int default_wall_thickness_px = 10;

/** Top HUD header height in pixels. */
const int ui_header_height = 20;

/** Maximum number of cookies allowed to be rendered on the screen at once. */
const int max_rendered_cookies = 20;

/** ----- Color Palette ----- */
const uint16_t color_cookie_dough = 0xE5CE; // CookieDough (0xFFE0B978)
const uint16_t color_cookie_golden = 0xDD09; // CookieGolden (0xD9A24A)
const uint16_t color_caramel_brown = 0xB385; // CaramelBrown (0xB5722E)
const uint16_t color_chocolate_chip = 0x59C4; // ChocolateChip (0x5A3A22)
const uint16_t color_dark_cocoa = 0x3902; // DarkCocoa (0x3B2317)
const uint16_t color_milk_cream = 0xFFDD; // MilkCream (0xFFFF8EC)
```

```
const uint16_t color_frosting_white = 0xFFFF; // FrostingWhite (0xFFFFDF8)
const uint16_t color_cinnamon = 0x8AC5; // Cinnamon (0x8C5A2B)
const uint16_t color_berry_red = 0xC245; // BerryRed (0xC0492F)
const uint16_t color_mint_green = 0x4C6B; // MintGreen (0x4C8C5A)
const uint16_t color_honey = 0xF5C9; // Honey (0xF2B84B)

/** ----- Main Game Mapping (Backward Compatible Names) ----- */
const uint16_t color_background = color_dark_cocoa; // Espresso Brown base
const uint16_t color_wall_type_1 = color_cookie_dough; // Golden Dough paths
const uint16_t color_wall_type_2 = color_cinnamon; // Cinnamon pathways
const uint16_t color_sphere = color_berry_red; // Jam Red Sphere
const uint16_t color_cookie = color_honey; // Honey Yellow dots
const uint16_t color_ui_text = color_milk_cream; // Warm milk HUD text

/** ----- Physics settings ----- */
/** IMU sensitivity multiplier (IMU = Inertial Measurement Units) */
const float default_imu_sensitivity_multiplier = 100.0f;

/** Bounce restitution coefficient.
 * This coefficient determines how much energy is conserved in a collision.
 * A value of 1 means a perfectly elastic collision (no energy loss), while
 * a value of 0 means a perfectly inelastic collision (objects stick together).
 * A value of 0.75 means that 75% of the energy is conserved in the collision,
 * which results in a realistic bounce effect for the ball in the game.
 * This value can be adjusted based on the desired game feel and physics
 * behavior.
 */
const float default_bounce_restitution = 0.3f;

/** Exponential moving average alpha.
 * This is the smoothing factor for the exponential moving average filter
 * applied to the sensor data.
 * A higher alpha gives more weight to recent data points, while a lower alpha
 * gives more weight to older data points.
 * new_value = alpha * new_measurement + (1 - alpha) * old_value
 * So, having a low alpha reduces the need for sudden movements in order to move
 * the ball.
 */
const float default_ema_alpha = 0.5f;

/** Deadzone threshold for sensor measurements.
 * This threshold defines the minimum change in sensor readings that will be
 * considered significant.
 * If the change in sensor readings is below this threshold, it will be ignored
 * to prevent noise from affecting the game.
 * This helps to create a more stable and enjoyable gaming experience by
 * filtering out small, insignificant
```

```
* movements of the device, e.g. when the player is holding the device still or
  making very slight movements.
*/
const float default_deadzone_threshold = 0.04f;

/** Maximum speed for the ball [px/s] to travel per second.*/
const float default_max_speed = 300.0f;

/** Continuous drag [1/s].
 * Default 0 means NO permanent damping, so the ball
 * never stalls on its own; energy is only lost on collisions.
 * Otherwise, a positive value will apply a continuous drag force to the ball,
 * gradually slowing it down over time.
 */
const float default_linear_damping = 0.0f;

/** Maximum number of contact resolution iterations per sub-step (corner safety)
  .*/
const int default_max_contact_iterations = 4;

/** Standard earth gravity in m/s^2. */
const float default_earth_gravity_g = 1.00f;

/** ----- Time settings ----- */
/** NTP server address 1 for time synchronization. */
const char *const ntp_server_1 = "pool.ntp.org";
/** NTP server address 2 for time synchronization. */
const char *const ntp_server_2 = "time.nist.gov";
/** GMT offset in seconds (0 for UTC). */
const long gmt_offset_sec = 0;
/** Daylight saving time offset in seconds (0 if not used). */
const int daylight_offset_sec = 0;
/** Unix epoch time for January 1, 2020 (used as a threshold for time
  synchronization). */
const int time_epoch_2020_Jan_1 = 1577836800;

/** ----- Hardware Pins ----- */
/** I2C SDA Pin for IMU and Touch. */
const int pin_i2c_sda = 11;
/** I2C SCL Pin for IMU and Touch. */
const int pin_i2c_scl = 10;
/** ADC Pin for Battery reading. */
const int pin_battery_adc = 1;
/** Boot Button Pin. */
const int pin_boot_button = 0;
/** PWR Button Pin. SYS_OUT pulled up, pulled low when pressed; SYS_EN controls
  battery power hold.*/
const int pin_power_button = 40;
```

```
/** ----- LCD Pins -----
 * The following pins are used for the SPI interface to communicate with the LCD
 * display.
 */
/** LCD Data/Command Pin. */
const int pin_lcd_dc = 4;
/** LCD Chip Select Pin. */
const int pin_lcd_cs = 5;
/** LCD Serial Clock Pin. */
const int pin_lcd_sck = 6;
/** LCD Master Out Slave In Pin. */
const int pin_lcd_mosi = 7;
/** LCD Reset Pin. */
const int pin_lcd_rst = 8;
/** LCD Backlight Pin (optional, can be used to control the backlight) */
const int pin_lcd_bl = 15;

/** ----- Analog & Battery Settings ----- */
/** MCU ADC Reference Voltage. */
const float adc_voltage_reference = 3.3f;
/** ESP32-S3 12-bit ADC max value (212 - 1). */
const float adc_max_resolution = 4095.0f;
/** Voltage divider multiplier for battery reading (1:1 resistor configuration
 * gives 2.0f). */
const float battery_voltage_multiplier = 2.0f;
/** Minimal simulated battery voltage. */
const float battery_min_voltage_mock = 3.0f;
/** Maximal simulated battery voltage. */
const float battery_max_voltage_mock = 4.2f;

/** ----- Concurrency and Multithreading Settings ----- */
/** Network core ID. */
#define core_network 0
/** Game core ID. */
#define core_game 1
/** Game loop tick rate in milliseconds. Equals 50Hz. */
#define game_tick_rate_ms 20
/** Telemetry publishing rate in milliseconds. Equals 2Hz. */
#define telemetry_rate_ms 500
/** FreeRTOS queue length. */
#define cmd_queue_length 10
/** FreeRTOS queue item size. */
#define cmd_queue_item_size sizeof(CommandMsg)

#endif // CONFIG_H
```

src/engine/GameConfig.h

Path: esp32/src/engine/GameConfig.h

```
#ifndef ENGINE_GAME_CONFIG_H
#define ENGINE_GAME_CONFIG_H

#include <stdint.h>
#include "../physics/core/PhysicsParams.h"
#include "../../config.h"

/**
 * @brief Transport-agnostic game configuration applied by the engine.
 *
 * Filled from an MQTT command (or defaults).
 */
struct GameConfig
{
    uint8_t gameId = default_game_id;          ///< 1 = Labyrinth, 2 =
        Flatland.
    uint16_t targetCookies = default_cookies_count; ///< Score needed to win a
        round.
    uint8_t visibleCookies = default_max_visible_cookies; ///< k cookies on
        screen at once.
    uint8_t wallThicknessPx = default_wall_thickness_px; ///< Maze block size (
        Game 1).
    PhysicsParams physics;                      ///< Tunable physics
        parameters.
};

#endif // ENGINE_GAME_CONFIG_H
```

src/engine/GameEngine.h

Path: esp32/src/engine/GameEngine.h

```
#ifndef ENGINE_GAME_ENGINE_H
#define ENGINE_GAME_ENGINE_H

#include <stdint.h>
#include "GameConfig.h"
#include "../physics/core/PhysicsEngine.h"
#include "../game/GameState.h"
#include "../game/modes/IGame.h"
#include "../game/modes/Game1Labyrinth.h"
#include "../game/modes/Game2Flatland.h"

/**
 * @brief Orchestrates both games behind one shared engine
 */
```

```
* The active game is selected via
* chooseGameMode(); the per-frame tick (input -> physics -> pickup -> telemetry
* meta) runs here so the games stay thin.
*
* Each game owns its OWN round counter; the engine only decides when to start
* vs. advance:
* - chooseGameMode(id): selects the game and start()s it -> the game's round
*   resets to 1 (so changing the mode selection always resets the count),
*   rebuilds the level (Game 1 -> new maze) and requests a full redraw.
* - nextRound(): advances the active game, which bumps its own counter (Game 2
*   counts rounds too) and rebuilds the level (Game 1 -> new maze).
*/
class GameEngine
{
public:
    /** @brief Allocates the shared play board and initializes both games. */
    void begin(int playWidth, int playHeight);

    /** @brief Applies configuration (physics params, cookie counts, etc.). */
    void applyConfig(const GameConfig &cfig);

    /** @brief Selects a game (1/2), resets the round to 1 and starts it. */
    void chooseGameMode(uint8_t gameId);

    /** @brief Advances to the next round of the active game (regenerates Game 1
        s maze). */
    void nextRound();

    /** @brief Stops the active game and drops the engine back to idle.
        * Needed so a STOP command actually sticks - otherwise update() keeps the
        * state RUNNING and the telemetry overwrites the idle flag again. */
    void stop();

    /** @brief Resumes a paused (idle) game right where it left off, keeping the
        * ball, cookies, round and timer. Unlike chooseGameMode() it does NOT
        * rebuild
        * the level. Does nothing if there's no game or it already finished. */
    void resume();

    /** @brief True when there's a paused game that resume() could pick back up.
        */
    bool canResume() const { return _active != nullptr && _status ==
        RunningStatus::IDLE; }

    /**
        * @brief Advances the active game by one frame.
        * @param tiltX In-plane tilt input (e.g. accelerometer X).
        * @param tiltY In-plane tilt input (e.g. accelerometer Y).
```

```
* @param dt    Time step [s].
*/
void update(float tiltX, float tiltY, float dt);

/** @brief Current scoreboard meta (runningStatus, round, score, time). */
const GameState &state() const { return _state; }

/** @brief Returns and clears the "needs full redraw" flag. */
bool consumeRedraw();

// Telemetry/ Status getter
const uint8_t *board() const { return _board; }
int playWidth() const { return _playW; }
int playHeight() const { return _playH; }
const PhysicsBody &ball() const { return _active->body(); }
const CookieField &cookies() const { return _active->cookies(); }
uint8_t activeGameId() const { return _active ? _active->id() : 0; }
uint16_t round() const { return _active ? _active->round() : 0; }
bool isRunning() const { return _status == RunningStatus::RUNNING; }

private:
    /** @brief Post-build step for chooseGameMode/nextRound: runningStatus +
        meta. */
    void afterBuild();

    PhysicsEngine _physics;
    GameConfig _config;
    GameState _state;

    Game1Labyrinth _game1;
    Game2Flatland _game2;
    IGame *_active = nullptr;

    uint8_t *_board = nullptr;
    int _playW = 0;
    int _playH = 0;

    float _elapsedSec = 0.0f;
    RunningStatus _status = RunningStatus::IDLE;
    bool _needsFullRedraw = false;
};

#endif // ENGINE_GAME_ENGINE_H

src/game/GameState.h

Path: esp32/src/game/GameState.h

#ifndef GAME_STATE_H
```



```
#define GAME_STATE_H

#include "RunningStatus.h"
#include "../../config.h"

/**
 * @struct GameState
 * @brief Holds volatile metadata used for rendering UI dashboard text.
 */
struct GameState
{
    RunningStatus runningStatus = RunningStatus::IDLE;
    int cookiesCollected = 0;
    int cookiesRemaining = default_cookies_count;
    int currentRound = 1;
    float elapsedTimeSec = 0.0f;
};

#endif // GAME_STATE_H
```

src/game/RunningStatus.h

Path: esp32/src/game/RunningStatus.h

```
#ifndef RUNNING_STATUS_H
#define RUNNING_STATUS_H

enum class RunningStatus
{
    IDLE,
    RUNNING,
    COMPLETED
};

#endif // RUNNING_STATUS_H
```

src/game/cookies/Cookie.h

Path: esp32/src/game/cookies/Cookie.h

```
#ifndef COOKIE_H
#define COOKIE_H

#include "../../config.h"

/**
 * @brief A single collectible cookie.
 *
 * @param x      Center X [px].
```

```
* @param y      Center Y [px].
* @param radius Collision radius [px].
* @param active True when the cookie is visible and can be collected.
*/
struct Cookie
{
    float x;
    float y;
    float radius;
    bool active; // false once collected (until it respawns)

    Cookie(float x = 0.0f, float y = 0.0f, float radius = default_cookie_radius,
           bool active = false)
        : x(x), y(y), radius(radius), active(active) {}
};

#endif // COOKIE_H
```

src/game/cookies/CookieField.h

Path: esp32/src/game/cookies/CookieField.h

```
#ifndef COOKIE_FIELD_H
#define COOKIE_FIELD_H

#include <stdint.h>
#include "Cookie.h"
#include "../../physics/core/PhysicsBody.h" // for PhysicsBody in checkPickup
#include "../../config.h"

// Max count of cookies
static const int kMaxVisibleCookies = default_max_visible_cookies;

/**
 * @brief Produces a fresh cookie at a valid position for the current game.
 *
 * Game 2 supplies a rectangle spawner, Game 1 a maze-cell spawner. CookieField
 * depends only on this interface, so the pickup/respawn logic stays shared and
 * unit-testable.
 */
class ICookieSpawner
{
public:
    virtual ~ICookieSpawner() = default;

    /**
     * @brief Returns a new active cookie at a valid position.
     * @param avoid A body the new cookie should not overlap (the ball).
     */
}
```

```
    virtual Cookie spawn(const PhysicsBody &avoid) const = 0;
};

/**
 * @brief A small set of cookies that respawn on contact.
 *
 * Keeps a few visible cookies (1-3). When the ball touches one, the score
 * increases and that cookie reappears at a new position from the injected
 * spawner. The game is won once collected() reaches target(). Reusable in both
 * games!
 * Only the CookieField itself knows the current score and target; the game loop
 * queries it via collected() and remaining().
 */
class CookieField
{
public:
    /**
     * @brief (Re)starts the field for a new round.
     *
     * @param visibleCount Cookies on screen at once (clamped to
     *         kMaxVisibleCookies).
     * @param target      Score needed to win (e.g. cmd.cookiesCount).
     * @param spawner     Position source. MUST outlive this CookieField.
     * @param ball        Current ball, so the initial cookies avoid it.
     */
    void start(uint8_t visibleCount, uint16_t target,
               const ICookieSpawner &spawner, const PhysicsBody &ball);

    /**
     * @brief Checks contact; on a hit, scores and respawns the eaten cookie.
     * @return Number of cookies eaten in this call (usually 0 or 1).
     */
    uint8_t checkPickup(const PhysicsBody &ball);

    uint16_t target() const { return _target; }
    uint16_t collected() const { return _collected; }
    uint16_t remaining() const { return (_target > _collected) ? (_target -
        _collected) : 0; }
    bool finished() const { return _collected >= _target; }

    uint8_t count() const { return _count; } // visible cookies (for rendering)
    const Cookie &at(uint8_t i) const { return _cookies[i]; }
    const Cookie *data() const { return _cookies; } // contiguous array (for
        rendering)

private:
    Cookie _cookies[kMaxVisibleCookies]; // fixed size -> no heap (MCU-friendly)
    uint8_t _count = 0;
};
```

```
uint16_t _target = 0;
uint16_t _collected = 0;
const ICookieSpawner *_spawner = nullptr; // owned by the game, must outlive
      us
};

#endif // COOKIE_FIELD_H
```

src/game/cookies/MazeCookieSpawner.h

Path: esp32/src/game/cookies/MazeCookieSpawner.h

```
#ifndef MAZE_COOKIE_SPAWNER_H
#define MAZE_COOKIE_SPAWNER_H

#include <Arduino.h> // for random()
#include <vector>
#include "CookieField.h" // ICookieSpawner, Cookie, PhysicsBody
#include "../../maze/MazeManager.h" // MazeManager::FreeCell
#include "../../config.h"

/**
 * @brief Game 1 spawner: places cookies only on carved maze corridors.
 *
 * The maze counterpart to RectCookieSpawner. Instead of sampling the whole
 * rectangle (which would drop cookies inside walls), it draws exclusively from
 * the corridor-cell centers exposed by MazeManager::getFreeCells(), so a cookie
 * can never land inside a wall.
 *
 * The cell list (owned by the MazeManager) MUST outlive this spawner.
 */
class MazeCookieSpawner : public ICookieSpawner
{
public:
    MazeCookieSpawner(const std::vector<MazeManager::FreeCell> *cells = nullptr,
                      float cookieRadius = default_cookie_radius)
        : _cells(cells), _r(cookieRadius) {}

    Cookie spawn(const PhysicsBody &avoid) const override
    {
        if (_cells == nullptr || _cells->empty())
        {
            return Cookie(0.0f, 0.0f, _r, false); // no maze -> inactive cookie
        }

        const std::vector<MazeManager::FreeCell> &cells = *_cells;
        const float minD = avoid.radius + _r + 2 * _r;

        Cookie c(0.0f, 0.0f, _r, true);
    }
};
```

```

    for (int t = 0; t < 20; ++t) // retry until far enough from the ball
    {
        const MazeManager::FreeCell &fc = cells[random((long)cells.size())];
        c.x = fc.x;
        c.y = fc.y;
        const float dx = c.x - avoid.x;
        const float dy = c.y - avoid.y;
        if (dx * dx + dy * dy >= minD * minD)
        {
            break;
        }
    }
    return c;
}

private:
    const std::vector<MazeManager::FreeCell> *_cells; // owned by MazeManager
    float _r;
};

#endif // MAZE_COOKIE_SPAWNER_H

```

src/game/cookies/RectCookieSpawner.h

Path: esp32/src/game/cookies/RectCookieSpawner.h

```

#ifndef RECT_COOKIE_SPAWNER_H
#define RECT_COOKIE_SPAWNER_H

#include <Arduino.h> // for random()
#include "CookieField.h"
#include "../config.h"

/**
 * @brief Game 2 spawner: a random position inside [0, width] x [0, height],
 * kept a margin away from the edges and away from the ball.
 */
class RectCookieSpawner : public ICookieSpawner
{
public:
    RectCookieSpawner(float width = 0.0f, float height = 0.0f, float
        cookieRadius = default_cookie_radius)
        : _w(width), _h(height), _r(cookieRadius) {}

    Cookie spawn(const PhysicsBody &avoid) const override
    {
        const float m = _r + 2.0f; // margin from the edges
        Cookie c(0.0f, 0.0f, _r, true);
        for (int t = 0; t < 20; ++t) // retry until far enough from the ball

```

```
{
    c.x = m + (random(1001) / 1000.0f) * (_w - 2.0f * m);
    c.y = m + (random(1001) / 1000.0f) * (_h - 2.0f * m);

    // TODO: Check if the usage of the default value is fine here.
    // Changed 3.0f to constant.
    const float dx = c.x - avoid.x;
    const float dy = c.y - avoid.y;
    const float minD = avoid.radius + _r + default_cookie_radius;

    if (dx * dx + dy * dy >= minD * minD)
    {
        break;
    }
}
return c;
}

private:
    float _w;
    float _h;
    float _r;
};

#endif // RECT_COOKIE_SPAWNER_H

src/game/modes/BaseGame.h

Path: esp32/src/game/modes/BaseGame.h

#ifndef GAME_BASE_GAME_H
#define GAME_BASE_GAME_H

#include "IGame.h"

/**
 * @brief Shared state and behavior for both game modes
 *
 * Holds the ball and the cookie field and implements the parts that are
 * identical between games (pickup, win check, accessors). Concrete games only
 * add their world/collider and spawner and implement id()/init()/startRound().
 */
class BaseGame : public IGame
{
public:
    // Round management is identical for every mode: selecting a mode restarts
    // it
    // at round 1, advancing bumps this game's own counter. Only buildLevel()
    // (the level/maze/cookie setup) differs per game.
```

```
void start(const GameConfig &cfg) override
{
    _round = 1;
    buildLevel(cfg);
}
void nextRound(const GameConfig &cfg) override
{
    _round++;
    buildLevel(cfg);
}
uint16_t round() const override { return _round; }

PhysicsBody &body() override { return _ball; }
const PhysicsBody &body() const override { return _ball; }
void onResolved() override { _cookies.checkPickup(_ball); }
bool finished() const override { return _cookies.finished(); }
const CookieField &cookies() const override { return _cookies; }

protected:
    /** @brief (Re)builds this game's level for the current round. */
    virtual void buildLevel(const GameConfig &cfg) = 0;

    uint16_t _round = 0; ///< This game's own round counter (0 until start()).
    PhysicsBody _ball;
    CookieField _cookies;
    uint8_t *_board = nullptr; ///< Engine-owned; not freed here.
    int _w = 0;
    int _h = 0;
    int _t = 1;
};

#endif // GAME_BASE_GAME_H

src/game/modes/Game1Labyrinth.h

Path: esp32/src/game/modes/Game1Labyrinth.h

#ifndef GAME_GAME1_LABYRINTH_H
#define GAME_GAME1_LABYRINTH_H

#include <vector>
#include "BaseGame.h"
#include "../cookies/MazeCookieSpawner.h"
#include "../../maze/MazeManager.h"
#include "../../physics/colliders/MazeCollider.h"
#include "../../config.h"

/**
 * @brief Game 1: randomly generated maze.
```

```
*
* Owns a MazeManager that regenerates a fresh maze into the shared board on
* every startRound(), a MazeCollider
* over that board, and the maze cookie spawner (k cookies respawn on corridor).
*/
class Game1Labyrinth : public BaseGame
{
public:
    Game1Labyrinth() : _maze(1, 1, 1) {}

    uint8_t id() const override { return 1; }

    void init(uint8_t *board, int playW, int playH, int wallThickness) override
    {
        _board = board;
        _w = playW;
        _h = playH;
        _t = wallThickness;
        _maze = MazeManager(playW, playH, wallThickness);
        _world = MazeCollider(board, playW, playH);
        // Spawner reads the maze's free-cell list, which is re-filled in place
        // on each generate(); the pointer stays valid across regenerations.
        // Cookie size scales with the corridor too (like the ball, just smaller)

        _spawner = MazeCookieSpawner(&_maze.getFreeCells(),
                                      wallThickness * cookie_diameter_wall_ratio /
                                      2.0f);
    }

    const ICollider &collider() const override { return _world; }

protected:
    void buildLevel(const GameConfig &cfg) override
    {
        _maze.generate(_board); // fresh maze for this round

        const std::vector<MazeManager::FreeCell> &cells = _maze.getFreeCells();
        if (!cells.empty())
        {
            _ball.x = cells[0].x; // start on a guaranteed corridor cell
            _ball.y = cells[0].y;
        }
        else
        {
            _ball.x = _w * 0.5f;
            _ball.y = _h * 0.5f;
        }
        _ball.vx = 0.0f;
    }
};
```



```
    _ball.vy = 0.0f;
    // Derive the ball size from the wall thickness so it always fits the
    // corridor: diameter = 70% of a corridor's width.
    _ball.radius = cfg.wallThicknessPx * ball_diameter_wall_ratio / 2.0f;

    _cookies.start(cfg.visibleCookies, cfg.targetCookies, _spawner, _ball);
}

private:
    MazeManager _maze;
    MazeCollider _world;
    MazeCookieSpawner _spawner;
};

#endif // GAME_GAME1_LABYRINTH_H

src/game/modes/Game2Flatland.h

Path: esp32/src/game/modes/Game2Flatland.h

#ifndef GAME_GAME2_FLATLAND_H
#define GAME_GAME2_FLATLAND_H

#include <string.h>
#include "BaseGame.h"
#include "../cookies/RectCookieSpawner.h"
#include "../../../physics/colliders/BorderCollider.h"
#include "../../../config.h"

/**
 * @brief Game 2: an open play-field limited only by the screen edges.
 *
 * Uses a BorderCollider and the rectangle cookie spawner (k cookies respawn on
 * pickup). No maze; the shared board is cleared to free space for rendering.
 */
class Game2Flatland : public BaseGame
{
public:
    Game2Flatland() : _world(1.0f, 1.0f) {}

    uint8_t id() const override { return 2; }

    void init(uint8_t *board, int playW, int playH, int wallThickness) override
    {
        _board = board;
        _w = playW;
        _h = playH;
        _t = wallThickness;
        _world = BorderCollider((float)playW, (float)playH);
    }
};
```

```
    }

    const ICollider &collider() const override { return _world; }

protected:
    void buildLevel(const GameConfig &cfg) override
    {
        if (_board != nullptr)
        {
            memset(_board, 0, (size_t)_w * _h); // open field: all free
        }
        _ball.x = _w * 0.5f;
        _ball.y = _h * 0.5f;
        _ball.vx = 0.0f;
        _ball.vy = 0.0f;
        // Open field has no corridor to scale to - use fixed, clearly visible
        sizes.
        _ball.radius = default_flatland_ball_radius;
        _spawner = RectCookieSpawner((float)_w, (float)_h,
            default_flatland_cookie_radius);
        _cookies.start(cfg.visibleCookies, cfg.targetCookies, _spawner, _ball);
    }

private:
    BorderCollider _world;
    RectCookieSpawner _spawner;
};

#endif // GAME_GAME2_FLATLAND_H

src/game/modes/IGame.h

Path: esp32/src/game/modes/IGame.h

#ifndef GAME_IGAME_H
#define GAME_IGAME_H

#include <stdint.h>
#include "../physics/core/PhysicsBody.h"
#include "../physics/colliders/ICollider.h"
#include "../cookies/CookieField.h"
#include "../engine/GameConfig.h"

/**
 * @brief A single game mode.
 *
 * Games carry ONLY what differs between modes: the collision world, where the
 * ball/cookies live, and how a round is (re)built. The shared per-frame tick
 * (input -> physics -> pickup -> telemetry) lives in GameEngine, so there is no
```

```
* duplicated game loop here.
*/
class IGame
{
public:
    virtual ~IGame() = default;

    /** @brief Stable id (1 = Labyrinth, 2 = Flatland). */
    virtual uint8_t id() const = 0;

    /**
     * @brief One-time setup: bind the shared play-field board and dimensions.
     * @param board      Engine-owned flat play board (width*height).
     * @param playW      Play-field width in pixels.
     * @param playH      Play-field height in pixels.
     * @param wallThickness Maze block size in pixels.
     */
    virtual void init(uint8_t *board, int playW, int playH, int wallThickness) =
        0;

    /**
     * @brief Starts the game at round 1 and builds the level.
     *
     * Called when this mode is selected, so the round counter resets to 1 every
     * time the game-mode selection changes. Game 1 regenerates the maze here.
     */
    virtual void start(const GameConfig &cfg) = 0;

    /**
     * @brief Advances this game to its next round and rebuilds the level.
     *
     * Increments the game's OWN round counter (each mode counts independently)
     * and regenerates the level (Game 1 -> a fresh maze).
     */
    virtual void nextRound(const GameConfig &cfg) = 0;

    /** @brief This game's current round number (1-based; 0 before start()). */
    virtual uint16_t round() const = 0;

    /** @brief The ball; engine integrates it against collider() each frame. */
    virtual PhysicsBody &body() = 0;
    virtual const PhysicsBody &body() const = 0;

    /** @brief Collision environment for this game's world. */
    virtual const ICollider &collider() const = 0;

    /** @brief Post-physics hook: collect cookies the ball now overlaps. */
    virtual void onResolved() = 0;
```

```
    /** @brief True once the win condition (all target cookies collected) is met
        . */
    virtual bool finished() const = 0;

    /** @brief Cookie field, for rendering and telemetry. */
    virtual const CookieField &cookies() const = 0;
};

#endif // GAME_IGAME_H
```

src/graphics/GraphicsManager.h

Path: esp32/src/graphics/GraphicsManager.h

```
#ifndef GRAPHICS_MANAGER_H
#define GRAPHICS_MANAGER_H

#include <Arduino.h>
#include <Arduino_GFX_Library.h>
#include "../config.h"
#include "../game/cookies/Cookie.h"
#include "../game/cookies/Cookie.h"
#include "../physics/core/PhysicsBody.h"
#include "../game/GameState.h"

/**
 * @class GraphicsManager
 * @brief Handles performance-optimized drawing, leveraging dirty tracking and
        partial updates.
 */
class GraphicsManager
{
public:
    /**
     * @brief Constructor for the Graphics Manager.
     * @param width Width of the display board in pixels.
     * @param height Height of the display board in pixels.
     */
    GraphicsManager(int width = display_width, int height = display_height);

    /**
     * @brief Attaches the physical or canvas-based GFX interface.
     * @param gfx Pointer to the Arduino_GFX display object.
     */
    void begin(Arduino_GFX *gfx);

    /**
```

```
* @brief Forces the engine to fully redraw the background and structures on
    the next loop tick.
*/
void forceFullRedraw();

/**
 * @brief The core rendering function executed every tick (50Hz).
 * Automatically switches between full updates and dynamic partial updates.
 *
 * @param mazeGrid Single-dimension bitmap array mapping states per pixel.
 * @param gridWidth Pixels wide of the active play area.
 * @param gridHeight Pixels high of the active play area.
 * @param ball Sphere object containing the current coordinate states.
 * @param cookies Array of all collectible objects.
 * @param cookieCount Dimension size of the cookies array.
 * @param state Scoreboard meta tracking details.
 */
void update(
    const uint8_t *mazeGrid,
    int gridWidth,
    int gridHeight,
    const PhysicsBody &ball,
    const Cookie *cookies,
    int cookieCount,
    const GameState &state);

/**
 * @brief Renders a stylized, brand-themed loading screen on the display.
 * Generates a warm espresso background with a detailed chocolate chip
    cookie
 * featuring procedural chips and a custom thematic "bite" out of its side.
 *
 * @param statusText Subtitle to display under the logo (e.g. "Loading...",
    "Connecting WiFi").
 */
void drawLoadingScreen(const char *statusText = "Loading...");

// Getters for internal tracking state (useful for validating within unit
    tests)
bool isFullRedrawNeeded() const { return _needsFullRedraw; }
PhysicsBody getPrevBallState() const { return _prevBall; }
int getWidth() const { return _width; }
int getHeight() const { return _height; }

private:
    int _width;
    int _height;
    Arduino_GFX *_gfx;
```

```
bool _needsFullRedraw;

// Local registers tracking changes between consecutive frames
PhysicsBody _prevBall;
bool _prevCookieActiveStates[max_rendered_cookies];
// Previous cookie positions, so a respawn (cookie stays active but moves)
// can be erased at the old spot and redrawn at the new one.
Cookie _prevCookies[max_rendered_cookies];
int _prevCookieCount;
GameState _prevGameState;

/**
 * @brief Draws the full play area pixel structures from scratch using
 *        horizontal line optimization.
 */
void drawFullMaze(const uint8_t *mazeGrid, int gridWidth, int gridHeight);

/**
 * @brief Redraws the HUD overlays selectively based on changed metrics.
 */
void drawUI(const GameState &state, bool forceDraw);

/**
 * @brief Restores the base maze states over localized bounding boxes to "
 *        erase" objects.
 */
void eraseRegion(int cx, int cy, int radius, const uint8_t *mazeGrid, int
    gridWidth, int gridHeight);

/**
 * @brief Draws a single circular cookie.
 */
void drawCookie(const Cookie &cookie);

/**
 * @brief Draws the sphere.
 */
void drawSphere(const PhysicsBody &ball);

/**
 * @brief Draws a "paused" symbol (two vertical bars, like a stopped video)
 *        centered over the play area. Shown while the game is idle/paused.
 */
void drawPauseOverlay();

/**
 * @brief Draws a "round complete" banner centered over the play area, shown
 *        for the brief moment between finishing a round and the next one starting.
```

```
        */
        void drawCompletedOverlay();
};

#endif // GRAPHICS_MANAGER_H

src/maze/MazeManager.h

Path: esp32/src/maze/MazeManager.h

#ifndef MAZE_MANAGER_H
#define MAZE_MANAGER_H

#include <stdint.h>
#include <vector>

/**
 * @class MazeManager
 * @brief Handles procedural generation of a maze using a randomized Depth-First
 *        Search (DFS) algorithm.
 *
 * This class slices the play-field into a block grid where each block is of
 * size wallThickness x wallThickness.
 * The active maze grid is mathematically centered within the play-field,
 * distributing any remainder pixel
 * padding symmetrically to the borders.
 *
 * The output is written into a flat 1D pixel array of size (width * height),
 * mapping pixels to:
 * - 0: Empty (passageway)
 * - 1: Wall Type 1 (outer borders / solid boundary padding)
 * - 2: Wall Type 2 (inner procedural maze walls)
 */
class MazeManager
{
public:
    /**
     * @struct FreeCell
     * @brief Pixel-space center of a DFS path node
     *
     * These are the only viable positions for cookies to be placed in the maze
     */
    struct FreeCell
    {
        float x; ///< Center X in pixels.
        float y; ///< Center Y in pixels.
    };

    /**
```

```
* @brief Constructs the MazeManager.
* @param width The width of the play area in pixels.
* @param height The height of the play area in pixels.
* @param wallThickness The thickness of walls and passages in pixels.
*/
MazeManager(int width, int height, int wallThickness);

/**
 * @brief Generates the procedural maze and centers it symmetrically within
 *        the provided flat buffer.
 * @param board Pre-allocated flat pixel buffer of size (width * height).
 * @return true if generation was successful, false otherwise.
 */
bool generate(uint8_t *board);

/**
 * @brief Returns the corridor-cell centers created by the last generate()
 *        call.
 *
 * Empty until generate() succeeds (also cleared when generation fails). Use
 * these as the only valid spawn positions inside the maze; see FreeCell.
 */
const std::vector<FreeCell> &getFreeCells() const { return _freeCells; }

/** @brief Returns the configured play-field width in pixels. */
int getWidth() const { return _width; }

/** @brief Returns the configured play-field height in pixels. */
int getHeight() const { return _height; }

/** @brief Returns the configured wall thickness in pixels. */
int getWallThickness() const { return _wallThickness; }

private:
    int _width;          ///< Width of the maze area in pixels.
    int _height;         ///< Height of the maze area in pixels.
    int _wallThickness;  ///< Thickness of the maze components in pixels.

    std::vector<FreeCell> _freeCells; ///< Corridor-cell centers from the last
    generate().

    /**
     * @struct Cell
     * @brief Coordinates of a pathway hub in the DFS grid.
     */
    struct Cell
    {
        int cx; ///< Column index in the cell grid.
```



```
    int cy; ///< Row index in the cell grid.

    bool operator==(const Cell &other) const
    {
        return cx == other.cx && cy == other.cy;
    }
};

/**
 * @brief Sets all pixels within a specific grid block to a target value,
 *        applying horizontal and vertical offsets.
 * @param board Flat pixel array.
 * @param gx Grid X coordinate in blocks.
 * @param gy Grid Y coordinate in blocks.
 * @param value The pixel state (0, 1, or 2).
 * @param offsetX Horizontal pixel offset to apply for centering.
 * @param offsetY Vertical pixel offset to apply for centering.
 */
void writeBlock(uint8_t *board, int gx, int gy, uint8_t value, int offsetX,
               int offsetY);

/**
 * @brief Records the pixel center of a carved corridor cell into _freeCells
 *        .
 * @param gx Grid X coordinate in blocks.
 * @param gy Grid Y coordinate in blocks.
 * @param offsetX Horizontal pixel offset applied for centering.
 * @param offsetY Vertical pixel offset applied for centering.
 */
void addFreeCell(int gx, int gy, int offsetX, int offsetY);

/**
 * @brief Fills the entire flat buffer with a target pixel value.
 * @param board Flat pixel array.
 * @param value The pixel state (0, 1, or 2).
 */
void fillBoard(uint8_t *board, uint8_t value);
};

#endif // MAZE_MANAGER_H
```

src/network/json-builder/JsonBuilder.h

Path: esp32/src/network/json-builder/JsonBuilder.h

```
#ifndef JSON_BUILDER_H
#define JSON_BUILDER_H
```

```
#include <Arduino.h>
```

```
#include <ArduinoJson.h>

/**
 * @brief Aggregates all telemetry data for a single transmission.
 *
 * Combines device information, game configuration, game state, physics
 * simulation
 * state, and sensor readings into a single structure. This decouples the
 * telemetry
 * serialization from the various components that provide the data.
 */
struct TelemetryData
{
    // Device Information
    String client_id;
    String hardware;
    String firmware_version;
    unsigned long uptime_ms;
    String wifi_ssid;

    // Game Configuration
    int game_id;
    String player_name;
    int target_cookies;
    int screen_width;
    int screen_height;
    int wall_thickness_px;

    // Game State
    String runningStatus;
    int cookies_collected;
    int cookies_remaining;
    int current_round;
    float elapsed_time_sec;

    // Physics Simulation State
    float ball_pos_x;
    float ball_pos_y;
    float velocity_x;
    float velocity_y;
    float acc_x;
    float acc_y;

    // Sensor Readings (from SensorData)
    float accel_x;
    float accel_y;
    float accel_z;
    float gyro_x;
```

```
    float gyro_y;
    float gyro_z;
    float battery_voltage;
    bool button;
};

/**
 * @brief Builds a telemetry JSON document from TelemetryData struct.
 *
 * Serializes the TelemetryData struct into a hierarchical JSON structure
 * compatible with the MQTT telemetry topic payload format.
 *
 * @param data The TelemetryData struct containing all telemetry values.
 * @return A serialized JSON string ready to publish via MQTT.
 */
String buildTelemetryJson(const TelemetryData &data);

/**
 * @brief Builds a test telemetry JSON document with mock data.
 *
 * Creates a complete telemetry payload with realistic sample values for testing
 * and development purposes.
 *
 * @return A serialized JSON string with test data ready for inspection or
 *         publishing.
 */
String buildTelemetryJsonTest();

#endif // JSON_BUILDER_H
```

src/network/json-parser/CommandParser.h

Path: esp32/src/network/json-parser/CommandParser.h

```
#ifndef COMMAND_PARSER_H
#define COMMAND_PARSER_H

#include <Arduino.h>
#include "../shared/CommandMsg.h"

/**
 * @brief Utility for decoding JSON payloads received via network channels.
 */
class CommandParser
{
public:
    /**
     * @brief Parses JSON string input command payloads into a CommandMsg.
     * @param jsonPayload Raw string input payload received from MQTT.
     */
};
```



```
* @param port    Broker port.
* @param clientId MQTT client identifier used during CONNECT.
* @param username Broker username.
* @param password Broker password.
* @param rootCa  PEM-encoded CA certificate used to validate the broker.
*
* @note All pointers must remain valid for the lifetime of the instance.
*/
MqttManager(const char *server,
            uint16_t port,
            const char *clientId,
            const char *username,
            const char *password,
            const char *rootCa);

/**
 * @brief Initialises the underlying MQTT client.
 *
 * Installs the CA certificate on the TLS transport, configures the broker
 * address, the keep-alive interval, the message callback and the publish
 * buffer size. Must be invoked once during device setup, after the system
 * clock has been synchronised so TLS certificate validation can succeed.
 */
void begin();

/**
 * @brief Establishes (or re-establishes) the connection to the broker.
 *
 * If the client is already connected the call is a no-op. Otherwise a
 * blocking connection attempt is performed using the configured
 * credentials and client identifier. On success a verification subscribe
 * and publish are issued against the test topic defined in config.h.
 *
 * @return true if the client is connected after the call, false otherwise.
 */
bool connect();

/**
 * @brief Drives the MQTT client's network loop.
 *
 * Must be invoked on every iteration of the Arduino loop() to service
 * incoming traffic and honour the configured keep-alive interval.
 */
void loop();

/**
 * @brief Indicates whether the MQTT client is currently connected.
 *

```

```
* @return true if connected, false otherwise.
*/
bool isConnected();

/**
 * @brief Publishes a payload to the given topic.
 *
 * @param topic Topic to publish on.
 * @param payload Payload to publish.
 * @param retain Retain flag forwarded to the broker.
 * @return true if the broker accepted the publish call, false otherwise
 * (for example when the client is not currently connected).
 */
bool publish(const char *topic, const String &payload, bool retain = false);

/**
 * @brief Subscribes to the given topic.
 *
 * @param topic Topic to subscribe to.
 * @param qos Requested quality of service (0 or 1).
 * @return true on success, false otherwise.
 */
bool subscribe(const char *topic, uint8_t qos = 0);

/**
 * @brief Disconnects the MQTT client from the broker.
 */
void disconnect();

/**
 * @brief Returns the underlying PubSubClient state code.
 *
 * @return The most recent PubSubClient state.
 */
int state();

private:
/**
 * @brief Static trampoline that forwards PubSubClient callbacks to the
 * currently active instance.
 *
 * PubSubClient does not portably support std::function, so a static
 * adapter is used to dispatch to a member function on the active instance.
 */
static void messageTrampoline(char *topic, byte *payload, unsigned int
    length);

/**
```

```
* @brief Handles a single message received on a subscribed topic.
*
* Logs the topic and payload length to the serial console. The payload is
* not null-terminated; the provided length must be respected when reading
* it.
*
* If the topic matches the configured command topic, the payload is parsed
* as a command message and the resulting CommandMsg is enqueued for the
* main loop to process using the shared state. Else the message is not
  further processed.
*/
void onMessage(char *topic, byte *payload, unsigned int length);

static MqttManager *_activeInstance;

WiFiClientSecure _secureClient;
PubSubClient _client;
const char *_server;
uint16_t _port;
const char *_clientId;
const char *_username;
const char *_password;
const char *_rootCa;
};

/**
 * @brief Shared MqttManager instance configured from wifi_mqtt_secrets.h and
 * config.h.
 *
 * Exposed so the main sketch and tests can drive MQTT through a single,
 * well-known entry point.
 */
extern MqttManager mqttManager;

#endif // MQTT_MANAGER_H

src/network/shared/CommandMsg.h

Path: esp32/src/network/shared/CommandMsg.h
#ifndef COMMAND_MSG_H
#define COMMAND_MSG_H

#include "CommandType.h"

/**
 * @brief Representation of an incoming dashboard command.
 * Contains configuration parameters and a unique request ID for tracking.
 */
```

```
struct CommandMsg
{
    CommandType type;
    char playerName[32];
    int gameId;
    int cookiesCount;
    int wallThicknessPx;
    float imuSensitivity;
    float bounceRestitution;
    float emaAlpha;
    float deadzoneThreshold;
    char requestId[36];
};
```

```
#endif // COMMAND_MSG_H
```

src/network/shared/CommandType.h

Path: esp32/src/network/shared/CommandType.h

```
#ifndef COMMAND_TYPE_H
#define COMMAND_TYPE_H

/**
 * @brief Enum representing the decoded command type.
 *
 * * START: Start the game with the provided parameters.
 * * STOP: Stop (pause) the game, keeping the current progress.
 * * RESUME: Continue a paused game right where it left off.
 * * PARAM_CHANGE: Update game parameters without starting or stopping the game.
 *   This is only optional for later use.
 * * UNKNOWN: Unknown command type.
 */
enum class CommandType
{
    START,
    STOP,
    RESUME,
    PARAM_CHANGE,
    UNKNOWN
};

#endif // COMMAND_TYPE_H
```

src/network/shared/MutexLock.h

Path: esp32/src/network/shared/MutexLock.h

```
#ifndef MUTEX_LOCK_H
```



```
#define MUTEX_LOCK_H

#include <Arduino.h>
#include <freertos/FreeRTOS.h>
#include <freertos/semphr.h>

/**
 * @brief RAII helper for locking and automatic unlocking of a FreeRTOS Mutex.
 * This class ensures that the mutex is properly released when the object goes
 * out of scope.
 */
class MutexLock
{
public:
    /**
     * @brief Constructs a MutexLock and attempts to acquire the given mutex.
     * @param mutex The FreeRTOS mutex handle to lock.
     * @param timeout The maximum time to wait for the mutex (default:
     * portMAX_DELAY).
     * @return None. The lock state can be checked with `isLocked()`.
     */
    explicit MutexLock(SemaphoreHandle_t mutex, TickType_t timeout =
        portMAX_DELAY)
        : m_mutex(mutex), m_locked(false)
    {
        if (m_mutex != nullptr)
        {
            if (xSemaphoreTake(m_mutex, timeout) == pdTRUE)
            {
                m_locked = true;
            }
        }
    }

    /**
     * @brief Destructor that releases the mutex if it was successfully acquired
     * .
     * This ensures that the mutex is always released when the MutexLock object
     * goes out of scope.
     */
    ~MutexLock()
    {
        if (m_locked && m_mutex != nullptr)
        {
            xSemaphoreGive(m_mutex);
        }
    }
}
```

```
/**
 * @brief Checks if the mutex was successfully acquired.
 * @return true if the mutex is currently locked by this MutexLock instance,
 *         false otherwise.
 */
bool isLocked() const { return m_locked; }

private:
    SemaphoreHandle_t m_mutex;
    bool m_locked;

    // Prevent copying and assignments
    MutexLock(const MutexLock &) = delete;
    MutexLock &operator=(const MutexLock &) = delete;
};

#endif // MUTEX_LOCK_H

src/network/shared/SharedStateData.h

Path: esp32/src/network/shared/SharedStateData.h

#ifndef SHARED_STATE_DATA_H
#define SHARED_STATE_DATA_H

#include <Arduino.h>
#include "src/game/RunningStatus.h"

/**
 * @brief Synchronized game state and configurations accessed by both CPU cores.
 * This structure is shared between the game logic and the network communication
 * tasks.
 */
struct SharedStateData
{
    // Identity (held so all dashboards show the same name; never defaulted)
    char playerName[32];

    // Parameters
    int cookiesCount;
    int wallThicknessPx;

    // Physics configs
    float imuSensitivity;
    float bounceRestitution;
    float emaAlpha;
    float deadzoneThreshold;

    // physics body state
```

```
float ballPosX;
float ballPosY;
float velocityX;
float velocityY;
float accX;
float accY;

// game state
int gameId;
int cookiesCollected;
int cookiesRemaining;
int currentRound;
float elapsedTimeSec;
RunningStatus runningStatus;
};

/**
 * @brief Initializer to safely set system defaults on start.
 * This function zeroes out the shared state and sets default values for all
 * fields.
 */
void initSharedState(SharedStateData &state);

#endif // SHARED_STATE_DATA_H
```

src/network/time/TimeManager.h

Path: esp32/src/network/time/TimeManager.h

```
#ifndef TIME_MANAGER_H
#define TIME_MANAGER_H

#include <Arduino.h>
#include <time.h>

/**
 * @brief Synchronises the MCU's internal clock with NTP and exposes the
 * current wall-clock time.
 *
 * An accurate clock is required for ISO 8601 timestamps on sensor data and
 * for validating the broker's TLS certificate during MQTT communication.
 * Configuration is supplied through the constructor so the helper can be
 * tested independently of the production configuration.
 */
class TimeManager
{
public:
    /**
     * @brief Constructs a TimeManager with the given NTP configuration.
```

```
*
* @param ntpServer1      Primary NTP server hostname.
* @param ntpServer2      Secondary NTP server hostname (fallback).
* @param gmtOffsetSec    Offset from UTC in seconds (0 for UTC).
* @param daylightOffsetSec Daylight-saving offset in seconds (0 if unused).
* @param epochThreshold Minimum unix epoch value that is considered a valid
*                        synchronised time.
*
* @note String pointers must remain valid for the lifetime of the instance.
*/
TimeManager(const char *ntpServer1,
            const char *ntpServer2,
            long gmtOffsetSec,
            int daylightOffsetSec,
            time_t epochThreshold);

/**
 * @brief Performs an NTP synchronisation and blocks until the system clock
 * holds a plausible wall-clock time.
 *
 * @warning A stable WiFi connection is required. The call blocks
 * indefinitely if no NTP response is ever received.
 */
void sync();

/**
 * @brief Returns the current time as a Unix timestamp.
 *
 * @return Seconds since the Unix epoch.
 */
time_t now() const;

/**
 * @brief Indicates whether the system clock currently holds a synchronised
 * time.
 *
 * @return true if the clock has been advanced past the configured threshold
 * ,
 * false otherwise.
 */
bool isSynchronized() const;

private:
    const char *_ntpServer1;
    const char *_ntpServer2;
    long _gmtOffsetSec;
    int _daylightOffsetSec;
    time_t _epochThreshold;
```

```
};

/**
 * @brief Shared TimeManager instance configured from config.h.
 */
extern TimeManager timeManager;

#endif // TIME_MANAGER_H

src/network/wifi-connection/WifiManager.h

Path: esp32/src/network/wifi-connection/WifiManager.h

#ifndef WIFI_MANAGER_H
#define WIFI_MANAGER_H

#include <Arduino.h>

/**
 * @brief Manages the device's WiFi connection lifecycle.
 *
 * Wraps the underlying WiFi driver and exposes a small, testable API for
 * connecting to a configured access point and querying connection state.
 * Credentials are supplied through the constructor so the class can be
 * unit-tested independently of the production configuration.
 */
class WifiManager
{
public:
    /**
     * @brief Constructs a WifiManager with the given credentials.
     *
     * @param ssid    Network SSID. Must remain valid for the lifetime of this
     *               instance.
     * @param password Network password. Must remain valid for the lifetime of
     *               this instance.
     */
    WifiManager(const char *ssid, const char *password);

    /**
     * @brief Establishes a connection to the configured WiFi network.
     *
     * Blocks until the connection is established. A stable WiFi connection is
     * required for the rest of the device to operate, so blocking here is
     * acceptable.
     *
     * @return true once the connection has been established.
     *
     * @warning If the credentials are incorrect or the network is unreachable

```

```
    * the call will block indefinitely.
    */
    bool connect();

    /**
     * @brief Indicates whether the device currently holds a WiFi connection.
     *
     * @return true if connected, false otherwise.
     */
    bool isConnected() const;

    /**
     * @brief Returns the IP address currently assigned by the access point.
     *
     * @return The local IP address as a string, or an empty string when not
     *         connected.
     */
    String localIP() const;

    /**
     * @brief Returns the SSID of the currently connected WiFi network.
     *
     * @return The SSID as a string, or an empty string when not connected.
     */
    String getSSID() const;

private:
    const char *_ssid;
    const char *_password;
};

/**
 * @brief Shared WifiManager instance configured from wifi_mqtt_secrets.h.
 *
 * Exposed so the main sketch and tests can drive WiFi connectivity through a
 * single, well-known entry point.
 */
extern WifiManager wifiManager;

#endif // WIFI_MANAGER_H

src/physics/colliders/BorderCollider.h

Path: esp32/src/physics/colliders/BorderCollider.h

#ifndef PHYSICS_BORDER_COLLIDER_H
#define PHYSICS_BORDER_COLLIDER_H

#include "ICollider.h"
```

```
/**
 * @brief Keeps a body inside an axis-aligned rectangle [0, width] x [0, height
 * ].
 *
 *
 * Used by Game 2.
 */
class BorderCollider : public ICollider
{
public:
    /**
     * @brief Constructs the border for a screen of the given size.
     * @param width Play-field width [px] (e.g. display_width).
     * @param height Play-field height [px] (e.g. display_height).
     */
    BorderCollider(float width, float height);

    Contact query(const PhysicsBody &body) const override;

private:
    float _width;
    float _height;
};

#endif // PHYSICS_BORDER_COLLIDER_H

src/physics/colliders/ICollider.h

Path: esp32/src/physics/colliders/ICollider.h

#ifndef PHYSICS_ICOLLIDER_H
#define PHYSICS_ICOLLIDER_H

#include "../core/Vec2.h"
#include "../core/PhysicsBody.h"

/**
 * @brief Result of a collision query for a body against an environment.
 */
struct Contact
{
    Vec2 normal;           // Normal vector pointing OUT of the obstacle (into free
                           // space).
    float penetration = 0; // Overlap depth [px]; >= 0 when hit.
    bool hit = false;      // True when the body overlaps an obstacle.
};

/**
 * @brief Strategy interface for the collision environment of a game.
```

```
*
*/
class ICollider
{
public:
    virtual ~ICollider() = default;

    /**
     * @brief Returns the deepest current contact for the given body.
     *
     * @param body Body to test against the environment.
     * @return Contact with hit == false when the body is in free space.
     */
    virtual Contact query(const PhysicsBody &body) const = 0;
};

#endif // PHYSICS_ICOLLIDER_H

src/physics/colliders/MazeCollider.h

Path: esp32/src/physics/colliders/MazeCollider.h

#ifndef PHYSICS_MAZE_COLLIDER_H
#define PHYSICS_MAZE_COLLIDER_H

#include <stdint.h>
#include "ICollider.h"

/**
 * @brief Collision environment backed by a MazeManager pixel board.
 *
 * Treats every non-zero board pixel (Wall Type 1 = border/padding, Wall Type 2
 * =
 * inner maze wall) as solid and every 0 pixel as free corridor. The outer board
 * ring is Wall Type 1, so this collider also keeps the ball inside the play
 * area; Game 1 needs no separate BorderCollider.
 *
 * Used by Game 1. The board pointer (owned by the engine / MazeManager) MUST
 * outlive this collider and is re-filled in place on every maze regeneration.
 */
class MazeCollider : public ICollider
{
public:
    /**
     * @param board Flat width*height pixel board (0 = free, !=0 = wall).
     * @param width Board width in pixels.
     * @param height Board height in pixels.
     */
    MazeCollider(const uint8_t *board = nullptr, int width = 0, int height = 0);
```



```
    Contact query(const PhysicsBody &body) const override;

private:
    const uint8_t *_board; ///< Not owned; must outlive this collider.
    int _width;
    int _height;
};

#endif // PHYSICS_MAZE_COLLIDER_H

src/physics/core/PhysicsBody.h
Path: esp32/src/physics/core/PhysicsBody.h
#ifndef PHYSICS_BODY_H
#define PHYSICS_BODY_H

#include "../../config.h"

/**
 * @brief Dynamic state of a single circular body (the ball).
 *
 * Every game has its own PhysicsBody, while the PhysicsEngine is shared!
 *
 * The radius of the Body is kept smaller than the corridor it passes through
 * for more precise movement
 */
struct PhysicsBody
{
    float x = 0.0f;           // Center position X [px], origin top
                             // -left.
    float y = 0.0f;           // Center position Y [px].
    float vx = 0.0f;          // Velocity X [px/s].
    float vy = 0.0f;          // Velocity Y [px/s].
    float radius = default_physics_body_radius; // Collision radius [px];
                             // smaller than the path width.
};

#endif // PHYSICS_BODY_H
```

```
src/physics/core/PhysicsEngine.h
Path: esp32/src/physics/core/PhysicsEngine.h
#ifndef PHYSICS_ENGINE_H
#define PHYSICS_ENGINE_H

#include "Vec2.h"
```

```
#include "PhysicsBody.h"
#include "PhysicsParams.h"
#include "../colliders/ICollider.h"
#include "../../config.h"

/**
 * @brief Shared, game-agnostic 2D physics engine.
 *
 * Holds the tuning parameters and the input filter state, and provides the
 * integration and collision-resolution algorithms.
 *
 * Energy is conserved except at collisions: there is no per-frame friction by
 * default, and a bounce only damps the velocity component normal to the wall,
 * so a ball scraping along a corridor keeps its speed along the path.
 */
class PhysicsEngine
{
public:
    /** @brief Replaces the active tuning parameters. */
    void setParams(const PhysicsParams &params);

    /** @brief Returns the active tuning parameters. */
    const PhysicsParams &params() const;

    /**
     * @brief Resets the input filter state.
     *
     * Should be called on restart or game switch to not carry over ema values
     */
    void reset();

    /**
     * @brief Conditions a raw tilt reading into an acceleration vector.
     *
     * Pipeline: EMA low-pass -> deadzone -> sensitivity scaling. The engine is
     * intentionally decoupled from SensorData; the game passes the in-plane
     * accelerometer components (e.g. accelerometerX / accelerometerY) directly.
     * This keeps the raw SensorData untouched for telemetry.
     *
     * @param tiltX Raw in-plane tilt input on X (e.g. accelerometer X).
     * @param tiltY Raw in-plane tilt input on Y.
     * @return Acceleration [px/s^2] to feed into step().
     */
    Vec2 inputAccel(float tiltX, float tiltY);

    /**
     * @brief Advances a body by one frame and resolves collisions.
     *
     */
}
```

```
* Uses semi-implicit Euler (velocity first, then position), clamps the
    speed
* to maxSpeed, and sub-steps the motion so a small ball cannot tunnel
* through a thin wall in a single frame. On contact only the normal
    velocity
* component is reflected and scaled by the restitution; the tangential
* component is preserved.
*
* @param body Body to advance (modified in place).
* @param accel Acceleration [px/s^2], typically from inputAccel().
* @param dt    Time step [s].
* @param world Collision environment.
* @return true if at least one collision was resolved this frame (maps to
*         the telemetry field physics_collision_detected).
*/
bool step(PhysicsBody &body, Vec2 accel, float dt, const ICollider &world);

private:
    PhysicsParams _params;
    float _emaX = 0.0f;
    float _emaY = 0.0f;
    bool _emaInitialized = false;
};

#endif // PHYSICS_ENGINE_H

src/physics/core/PhysicsParams.h

Path: esp32/src/physics/core/PhysicsParams.h

#ifndef PHYSICS_PARAMS_H
#define PHYSICS_PARAMS_H

#include "../../config.h"

/**
 * @brief Tunable physics parameters.
 *
 * These parameters can be adjusted via MQTT on runtime
 */
struct PhysicsParams
{
    // The "imu_sensitivity_multiplier" config setting; more granular adjustment
    // to movement sensitivity
    float sensitivity = default_imu_sensitivity_multiplier;

    // Bounce Factor applied to the object; 0 means no bounce; 1 means perfect
    // bounce
    float restitution = default_bounce_restitution;
};
```

```
// EMA smoothing alpha; Higher means more responsive
float emaAlpha = default_ema_alpha;

// Input deadzone: tilt below this value will not be registered
float deadzone = default_deadzone_threshold;

// The maximum value the ball can travel per second [px/s]
float maxSpeed = default_max_speed;

/** Continuous drag [1/s]. Default 0 means NO permanent damping, so the ball
 * never stalls on its own; energy is only lost on collisions. */
float linearDamping = default_linear_damping;
};

#endif // PHYSICS_PARAMS_H
```

src/physics/core/Vec2.h

Path: esp32/src/physics/core/Vec2.h

```
#ifndef PHYSICS_VEC2_H
#define PHYSICS_VEC2_H

/**
 * @brief Minimal 2D vector used throughout the physics engine.
 *
 * @param x X component.
 * @param y Y component.
 */
struct Vec2
{
    float x;
    float y;

    Vec2(float x = 0.0f, float y = 0.0f) : x(x), y(y) {}
};

#endif // PHYSICS_VEC2_H
```

src/sensors/SensorManager.h

Path: esp32/src/sensors/SensorManager.h

```
#ifndef SENSOR_MANAGER_H
#define SENSOR_MANAGER_H

#include <Arduino.h>
```

```
#include "../..//config.h"
#include "SensorQMI8658.hpp"

/**
 * @brief Aggregates a single snapshot of all sensor readings.
 *
 * Keeping the raw values in a dedicated struct decouples the sensor acquisition
 * code from the consumers that serialise or evaluate the data.
 *
 * @note The exact set of fields can be adjusted as additional sensors are
 * integrated into the project.
 */
struct SensorData
{
    String timestamp;
    float accelerometerX;
    float accelerometerY;
    float accelerometerZ;
    float gyroscopeX;
    float gyroscopeY;
    float gyroscopeZ;
    float batteryVoltage;
    bool button;
};

/**
 * @brief Owns the sensor peripherals and produces SensorData snapshots.
 *
 * Encapsulates the initialisation and read paths for all on-board sensors so
 * that the rest of the application can interact with sensor data through a
 * single, testable component.
 */
class SensorManager
{
public:
    /**
     * @brief Constructs a SensorManager. No hardware is touched until begin()
     * is called.
     */
    SensorManager();

    /**
     * @brief Initialises the sensor peripherals.
     *
     * Configures pin modes and performs any sensor-specific setup
     * (for example calibration) required before the first read().
     */
}
```

```
* The manager is considered initialised after core GPIO/ADC setup, even
  when
* the IMU is temporarily unavailable. In that case read() will safely fall
* back to the previously known IMU values until the device provides data.
*/
void begin();

/**
 * @brief Reads the current value of every sensor.
 *
 * @return SensorData snapshot containing the latest readings.
 *
 * @note begin() must have been invoked before the first call.
 */
SensorData read();

/**
 * @brief Mock implementation of read() for testing purposes.
 *
 * @return Random/Mocked SensorData snapshot.
 */
SensorData readMock();

/**
 * @brief Returns the last read SensorData snapshot.
 *
 * @return SensorData snapshot containing the most recent readings.
 */
SensorData getLastData() const
{
    return _lastData;
}

/**
 * @brief Indicates whether begin() completed manager setup.
 *
 * @return true once basic sensor manager setup has run, false otherwise.
 */
bool isInitialized() const;

private:
    bool _initialized;
    bool _imuAvailable;
    SensorData _lastData;

    // Hardware components
    SensorQMI8658 _qmi;
};
```

```
extern SensorManager sensorManager;  
  
#endif // SENSOR_MANAGER_H
```

References

- [1] AMAZON WEB SERVICES: *The Difference Between SSL and TLS*. <https://aws.amazon.com/compare/the-difference-between-ssl-and-tls/>. Version: 2026, Abruf: 30.06.2026
- [2] ARDUINO: *Arduino Language Reference*. <https://docs.arduino.cc/language-reference/>. Version: 2026, Abruf: 17.05.2026
- [3] COMPUTOOLS: *SPI Calculator*. <https://www.compu-tools.com/spi/>. Version: 2026, Abruf: 30.06.2026
- [4] CPPREFERENCE.COM: *Resource Acquisition Is Initialization (RAII)*. <https://en.cppreference.com/w/cpp/language/raii>. Version: 2026, Abruf: 28.06.2026
- [5] ESPRESSIF SYSTEMS: *ESP-IDF FreeRTOS API Reference*. <https://docs.espressif.com/projects/esp-idf/en/v4.3/esp32/api-reference/system/freertos.html>. Version: 2026, Abruf: 28.06.2026
- [6] GEEKSFORGEEKS: *Inter Process Communication (IPC)*. <https://www.geeksforgeeks.org/operating-systems/inter-process-communication-ipc/>. Version: 2026, Abruf: 28.06.2026
- [7] GOOGLE: *Compose Material 3*. <https://developer.android.com/jetpack/androidx/releases/compose-material3>. Version: 2026, Abruf: 30.06.2026
- [8] GOOGLE: *Save simple data with SharedPreferences*. <https://developer.android.com/training/data-storage/shared-preferences>. Version: 2026, Abruf: 30.06.2026
- [9] HIVEMQ: *HiveMQ*. <https://www.hivemq.com/>. Version: 2026, Abruf: 30.06.2026
- [10] HIVEMQ: *HiveMQ MQTT Client*. <https://hivemq.github.io/hivemq-mqtt-client/>. Version: 2026, Abruf: 30.06.2026
- [11] JETBRAINS: *Kotlin Programming Language*. <https://kotlinlang.org/>. Version: 2026, Abruf: 30.06.2026
- [12] JSON.ORG: *Introducing JSON*. <https://www.json.org/json-en.html>. Version: 2026, Abruf: 28.06.2026
- [13] KLINKER, Frank: Exponential moving average versus moving exponential average. In: *Mathematische Semesterberichte* 58 (2011), Nr. 1, S. 97–107
- [14] KNIGHT, Julian ; TOTALLY INFORMATION: *node-red-contrib-uibuilder*. <https://github.com/TotallyInformation/node-red-contrib-uibuilder/>
- [15] KNIGHT, Julian ; TOTALLY INFORMATION: *UIBUILDERv7 for Node-RED*. <https://totallyinformation.github.io/node-red-contrib-uibuilder/#/>. Version: 2026, Abruf: 17.05.2026
- [16] MICROSOFT: *Model View ViewModel (MVVM)*. <https://learn.microsoft.com/de-de/dotnet/architecture/maui/mvvm>. Version: 2026, Abruf: 30.06.2026

- [17] MOON ON OUR NATION: *Arduino_GFX_Library*. https://github.com/moononournation/Arduino_GFX. Version: 2026, Abruf: 28.06.2026
- [18] MOZILLA: *WebSocket API*. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API. Version: 2026, Abruf: 30.06.2026
- [19] MQTT.ORG: *MQTT: The Standard for IoT Messaging*. <https://mqtt.org/>. Version: 2026, Abruf: 28.06.2026
- [20] NACERKROUDIR: *Randomized Depth-First-Search Algorithm for Maze Generation*. <https://medium.com/@nacerkroudir/randomized-depth-first-search-algorithm-for-maze-generation-fb2d83702742>. Version: 2026, Abruf: 30.06.2026
- [21] NODE-RED DASHBOARD TEAM: *Node-RED Dashboard (deprecated)*. <https://flows.nodered.org/node/node-red-dashboard>. Version: 2026, Abruf: 17.05.2026
- [22] OPENJS FOUNDATION: *Node-RED – Low-code programming for event-driven applications*. <https://nodered.org>. Version: 2026, Abruf: 17.05.2026
- [23] PETERSON, Larry L. ; DAVIE, Bruce S.: *Computer Networks: A Systems Approach*. 6th. Morgan Kaufmann, 2023. <http://dx.doi.org/10.1016/C2018-0-01477-2>. <http://dx.doi.org/10.1016/C2018-0-01477-2>. – ISBN 978-0-12-818200-0
- [24] RUPERT, Dave: *Animated SVG Radial Progress Bars*. <https://daverupert.com/2018/03/animated-svg-radial-progress-bars/>. Version: 2026, Abruf: 30.06.2026
- [25] SHARDA, Aryaman: *Understanding How UUIDs Are Generated*. <https://digitalbunker.dev/understanding-how-uuids-are-generated/>. Version: 2020, Abruf: 30.06.2026
- [26] STUDYRAID: *Using Dirty Rect Animation Techniques*. <https://app.studyraid.com/en/read/15006/518776/using-dirty-rect-animation-techniques>. Version: 2025, Abruf: 29.06.2026
- [27] TRNKA, O. ; HARTMAN, M. ; SVOBODA, K.: An alternative semi-implicit Euler method for the integration of highly stiff nonlinear differential equations. In: *Computers & Chemical Engineering* 21 (1997), Nr. 3, 277-282. [http://dx.doi.org/https://doi.org/10.1016/S0098-1354\(95\)00266-9](http://dx.doi.org/https://doi.org/10.1016/S0098-1354(95)00266-9). – DOI [https://doi.org/10.1016/S0098-1354\(95\)00266-9](https://doi.org/10.1016/S0098-1354(95)00266-9). – ISSN 0098-1354
- [28] W3SCHOOLS: *HTML Canvas Graphics*. https://www.w3schools.com/html/html5_canvas.asp. Version: 2026, Abruf: 30.06.2026
- [29] WANG, Jui-Hsien ; SETALURI, Rajsekhar ; JAMES, Doug L. ; PAI, Dinesh K.: Bounce maps: an improved restitution model for real-time rigid-body impact. In: *ACM Trans. Graph.* 36 (2017), Nr. 4, S. 150–162
- [30] WAVESHARE INTERNATIONAL LIMITED: *ESP32-S3 1.69inch Touch Display Development Board*. <https://docs.waveshare.com/ESP32-S3-Touch-LCD-1.69>. Version: 2026, Abruf: 17.05.2026

- [31] WAVESHARETEAM: *ESP32-S3-Touch-LCD-1.69 Pin Configuration*. https://github.com/waveshareteam/ESP32-S3-Touch-LCD-1.69/blob/main/examples/Arduino/libraries/Mylibrary/pin_config.h. Version:2026, Abruf: 28.06.2026